



Universitat Autònoma de Barcelona

Master's Degree in Modelling for Science and Engineering

FINAL MASTER'S THESIS

Performance Optimization of the AI4Land Training Pipeline on MareNostrum 5

Author:
Ferran Mirabent Rubinat

Tutors:
Oriol Tintó Prims (BSC)
Sílvia Cuadrado Gavilán (UAB)

June 20, 2026

Summary

The AI4Land project at the Barcelona Supercomputing Center trains a deep-learning model to close a gap in climate data. Global records of how land is used reach back to 1850 and forward to 2100, but only at a coarse resolution of about thirty kilometres per pixel; the high-resolution maps that the next generation of climate models needs, at one kilometre per pixel, exist only for recent decades. The model sharpens the coarse record to one kilometre across the full time span. Training it on the MareNostrum 5 supercomputer was slow, and a first measurement showed the graphics processors, the most expensive part of the machine, sitting idle more than ninety per cent of the time while they waited for data.

This thesis explains why and removes the cause. Working in a loop of profiling a real run, changing one thing, and measuring the effect, the slow part is traced to how the training data is stored. The data sits in a very large number of small files on the cluster's shared filesystem, and the pipeline opens about ninety of them for every training sample. The expense is not in reading the bytes, which are few, but in the time the filesystem spends locating each file before any of it can be read. Storing the same data in far fewer, larger files cuts the opens per sample from about ninety to about nine, while leaving the data the model sees bit-for-bit identical. A single sample then loads about seven times faster, and a full training run finishes roughly 4.6 times sooner, with the gain holding as the work is spread across more processors. The long, periodic stalls that interrupted every earlier run disappear. The limit on training speed moves off the data pipeline and onto the graphics processors themselves, which is where it belongs.

The wider result is transferable. For any deep-learning pipeline that reads image patches from chunked geospatial storage on a supercomputer's shared filesystem, the way those files are laid out is the single largest factor in how fast it trains. The measurement tools and the layout principle developed here carry over directly to other projects, including the ELLIOT foundation-model effort that plans to reuse this pipeline.

Acknowledgments

I am grateful to Amirpasha Mozaffari, Marina Castaño, Amanda Duarte and Jordi Varela from the AI4Land team at the Barcelona Supercomputing Center for welcoming me into the project, sharing the codebase and the running pipeline, and patiently answering every question I had on the model, the data, and the cluster. I also want to thank Joan Vedri, who built the pipeline that generates the new datasets, helped me design the merged store at the centre of this work. and reviewed the code I wrote for the final dataset.

I would like to thank my tutors, Oriol Tintó Prims at BSC and Sílvia Cuadrado Gavilán at UAB, for the trust they placed in me, the freedom they gave me to explore the problem on my own terms, and the guidance they provided at every step.

Finally, I would like to thank my girlfriend, my family, and my friends for their unwavering support throughout this journey.

Contents

Summary	i
Acknowledgments	ii
1 Introduction	1
2 Objectives	6
3 Methodology	7
3.1 Literature Review	7
3.2 Tooling	7
3.3 The Current Training Pipeline	10
3.4 The U-Net Architecture	12
3.5 Optimization Loop and Measurement	13
4 Results	15
4.1 Establishing a Reproducible Workflow	15
4.1.1 Conda limitations and the move to uv	15
4.1.2 Code style and tests	16
4.1.3 Scientific and workflow impact	17
4.2 Initial Profiling	17
4.3 From Multiple Stores to a Single Store	19
4.4 Removing the Synchronizer	22
4.5 Isolating the Dataloader	23
4.6 Diagnosing the Bottleneck on Synthetic Data	24
4.6.1 Anatomy of a single chunk read	24
4.6.2 Per-sample bytes and output tensor size	25
4.6.3 Bandwidth ceiling and the observed gap	26
4.6.4 Metadata cost as a function of file count	26
4.6.5 Production-shape simulation	27
4.6.6 Cause of the store-size effect on read performance	28
4.6.7 Summary of evidence	29
4.7 From a Single Store to a Merged Store	30
4.7.1 Design of the merged store	30
4.7.2 Parity check	31
4.7.3 Per-sample performance	31
4.7.4 Per-step breakdown of the merged per-sample fetch	32
4.7.5 End-to-end training	32
4.7.6 Scale-out behaviour	33
4.8 Projected speed-up over the multi-zarr baseline	35
5 Conclusions	37
6 Future Work	38

1 Introduction

An Earth System Model, abbreviated ESM throughout the rest of this thesis, is a numerical simulator that represents the coupled evolution of the atmosphere, the ocean, the sea ice, the land surface, and the biogeochemical cycles that link them, on a discretized representation of the planet. The model solves the relevant fluid-dynamic and thermodynamic equations on a grid that covers the globe, with a finite time step, and it tracks how energy, water, and carbon move between those components over years, decades, or centuries. ESMs are the instruments that the climate community uses to produce *climate projections*: given an assumed trajectory of greenhouse-gas emissions, an ESM predicts how surface temperature, precipitation, sea level, atmospheric circulation, vegetation, and many other quantities will evolve over the next century. The projections that inform the assessment reports of the Intergovernmental Panel on Climate Change (IPCC) are produced by ESMs.

The land surface plays a substantial role in those projections. Vegetation cover, soil moisture, surface roughness, and human land-management practices regulate how much sunlight the surface absorbs, how much water evaporates back to the atmosphere, and how much carbon is exchanged between the biosphere and the atmosphere. The literature has identified land-related processes as a leading source of uncertainty in the projected terrestrial carbon cycle [8, 17]: depending on how the land surface is represented inside the ESM, different models can disagree on whether the land acts as a net carbon sink or as a net carbon source by the end of the twenty-first century, and that disagreement propagates into divergent surface-temperature projections.

To represent the land surface in a way that is useful for climate simulation, an ESM needs a spatially explicit description of *land use* and *land cover*, jointly referred to by the acronym LULC. Land cover refers to the physical material present at the surface (forest, grassland, bare soil, urban concrete, water); land use refers to how humans manage that surface (cropland, pasture, managed forest, urban infrastructure, protected area). The two concepts overlap but are not identical: a single land-cover class such as “grassland” can be associated with several different land uses, and the management practice often matters more for the climate than the cover class alone. ESMs ingest LULC information as a *boundary condition*, that is, as a time-dependent input that the model takes as externally given rather than predicting on its own.

The most widely used LULC boundary condition for centennial-scale climate simulations is the Land-Use Harmonization version 2 dataset (LUH2) [21]. LUH2 was produced for the sixth Coupled Model Intercomparison Project (CMIP6), the coordinated experimental protocol under which the simulations used by the most recent IPCC assessment were run. LUH2 provides fractional cover for twelve land-use categories on a global grid at a horizontal resolution of approximately thirty kilometres per pixel, sampled annually from the year 850 to 2100. The future portion of the record is not a single trajectory: it consists of several alternative continuations, each associated with one of the Shared Socioeconomic Pathways (SSPs). The SSPs are scenarios constructed by the climate-research community that span plausible combinations of demographic, economic, technological, and policy assumptions; each SSP yields a distinct projected evolution of the land surface, which the ESM then ingests as its land-side boundary condition.

The next generation of ESMs aims to resolve land/atmosphere coupling at horizontal

scales of the order of one kilometre per pixel, fine enough to capture city-scale urban heat islands, individual agricultural regions, coastal margins, and local hydrological catchments. At that target resolution, LUH2 is too coarse: a single LUH2 pixel covers an area of roughly nine hundred square kilometres, larger than many countries’ major metropolitan regions and more than three orders of magnitude coarser, in area, than what the next-generation ESMs require. Conversely, the high-resolution products that do exist, such as HILDA+ [40] (Historic Land Dynamics Assessment, version +), provide global land-use maps at one-kilometre resolution but only for the satellite era, roughly from 1960 onwards in the case of HILDA+. They contain no record of how the surface looked in 1850 and no projection of how it will look in 2100.

The mismatch between the temporally complete but spatially coarse LUH2 record and the spatially fine but temporally limited HILDA+ record is what motivates the AI4Land project, and will be referred to in the rest of this thesis as the *resolution gap*. Closing the gap means producing a single dataset, at the high spatial resolution that the next-generation ESMs demand, over the long temporal window that climate projections require.

The AI4Land project, developed at the Barcelona Supercomputing Center (BSC), addresses the resolution gap with a *deep learning* model. “Deep learning” is the name given to a family of statistical models, called artificial neural networks, in which a function from an input vector to an output vector is built up from many layers of simple operations (typically linear matrix multiplications followed by element-wise nonlinear activation functions). The free parameters of those operations (the entries of the matrices and the bias vectors, collectively called the *weights*) are tuned by minimizing a loss function on a training dataset, using stochastic gradient descent and one of its variants such as Adam. The minimization process is called *training* the model, and a single full pass through the training dataset is called an *epoch*. Once a model is trained, it is *evaluated* on data it has not seen during training (the *test set*) to verify that it generalizes, that is, that it has learned a function rather than memorized the training examples.

The specific deep-learning architecture used by AI4Land is a U-Net [34]. A U-Net is a particular kind of convolutional neural network whose layout resembles the letter “U”: the input image is progressively downsampled through a series of convolution and pooling operations, compressing spatial detail into more abstract features at coarser resolution (the encoder branch); the resulting compact representation is then progressively upsampled back to the input resolution, with direct “skip” connections that re-inject the encoder’s finer features at each scale (the decoder branch). U-Nets were originally introduced for biomedical image segmentation and have since become a standard choice for any task that maps an input image to an output of the same spatial dimensions. AI4Land’s task fits that template: the input is a multi-channel spatial map of climate forcings, and the output is a spatial map of predicted land-use class labels.

The task itself is framed as *semantic segmentation*. In a semantic segmentation problem, the model assigns a class label to every pixel of the input image, producing an output map of the same spatial size as the input but with one categorical channel per pixel instead of the original input channels. For AI4Land, the per-pixel classes are the consolidated land-use categories (forest, cropland, pasture, grassland or shrubland, urban, water, other), and the output is a land-use class for every one-kilometre pixel of the globe at a given year. The fact that the inputs and the outputs are spatial maps, rather than scalars or fixed-length vectors, is what makes a U-Net the appropriate choice of architecture.

The inputs that AI4Land fuses go beyond LUH2. Each one-kilometre pixel is described by a multi-channel feature vector that combines four families of information. The first is the coarse LUH2 forcing for the year in question, upsampled to the target one-kilometre grid. The second is a set of static geophysical priors that do not change year to year, including elevation, slope, and aspect derived from the GEBCO 2024 global elevation and bathymetry dataset [16] (the General Bathymetric Chart of the Oceans, a community-curated global ocean-bathymetry and land-elevation grid), and clay, sand, and organic content extracted from the Pacific Northwest National Laboratory’s (PNNL) global one-kilometre soil parameter dataset [23]. The third is a categorical climate-zone descriptor from the Köppen–Geiger classification [6], a climatological scheme that splits the globe into several dozen zones based on temperature and precipitation regimes (tropical, arid, temperate, continental, polar, and their finer-grained sub-zones). The fourth is a human-pressure proxy from the ISIMIP2b population dataset [15] (Inter-Sectoral Impact Model Intercomparison Project, phase 2b), which provides a global gridded population record for the historical period and for the same SSP scenarios that LUH2 projects forward.

A further design choice that turns out to be important for the downscaling task is the use of an *autoregressive prior*. When the model is asked to predict the land-use map for year t , it is also given the model’s own predicted map for the preceding year $t - 1$ as an additional input channel. This information is fed through a masking strategy that hides part of the prior map at random during training, so the model cannot learn to simply copy the prior at inference time. The autoregressive design propagates temporal coherence forward across the rollout: the prediction at year t is not independent of the prediction at year $t - 1$, which keeps the downscaled record temporally smooth across a centennial-scale window. The training methodology, the evaluation protocol, and the global inference results of the AI4Land model are described in detail in the project’s published and submitted papers [27, 28].

The project itself lives inside two Horizon Europe programmes funded by the European Commission. The first is CONCERTO [9], a project that works to reduce the land-driven uncertainty in Earth System Models by improving how the terrestrial carbon cycle is monitored and modelled, combining Earth observation, data assimilation, and machine learning. The second is TerraDT [37], a programme that constructs a “digital twin” of the Earth system focused on the cryosphere, the land surface, and their interactions; a digital twin in this context is a high-fidelity, continuously updated simulator of the real system, intended to support what-if analysis and decision-making. A third Horizon Europe programme, ELLIOT [13], plans to adopt the AI4Land training pipeline for training large geospatial *foundation models*, which are general-purpose deep-learning models trained once on a very large and heterogeneous corpus of satellite and climate data and then reused across many downstream prediction tasks. The optimization work in this thesis is therefore not consumed by AI4Land alone: any improvement in the pipeline propagates immediately to the training pipeline and tooling that ELLIOT will build on top of it.

Optimizing a training pipeline is not, in itself, a research goal: the science it enables is. But on a system like MareNostrum 5, allocations are finite, contention with other users is real, and the cost of a wasted node-hour (measured in money paid for the allocation, in time spent queueing for it, and in delayed iteration on the science the pipeline supports) is non-trivial. A pipeline that spends approximately ninety-nine per cent of its time waiting on data, as the unmodified AI4Land pipeline does during a representative profiling window, is a pipeline whose investment in H100 graphics processing units (GPUs) is being wasted: the

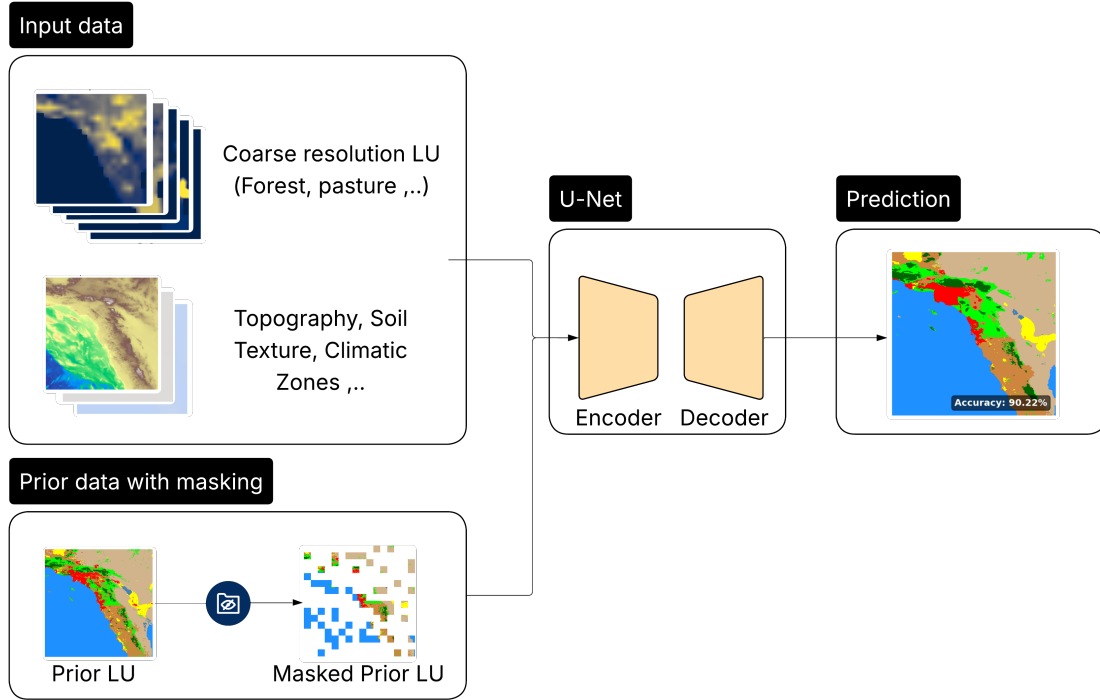


Figure 1: AI4Land reconstruction pipeline, reproduced from [28]. The U-Net backbone fuses coarse dynamic LUH2 variables with high-resolution static priors. A randomly masked land-use map from the adjacent time step is injected as autoregressive prior.

most expensive resource in the system is sitting idle while the cheaper resources are kept busy. Optimization here serves three concrete purposes. First, it shortens the research cycle, allowing more independent experiments per allocation and faster turnaround on each one. Second, it enables the model to see more of the world in the same wall-clock budget (more spatial coverage, more temporal range, or both). Third, it produces practical guidelines that are transferable to other scientific deep-learning pipelines on high-performance computing (HPC) infrastructure, which is exactly the kind of contribution the ELLIOT project will reuse.

Optimization in this regime is also closer to art than to science. Bottlenecks shift between layers as each is addressed, moving from the storage and input/output path to the central processing unit (CPU), the network, and the GPU’s memory and compute units. The tuning knobs interact non-linearly, so that a change which helps under one set of conditions can have no effect or even hurt under another. The same change can also yield very different speedups on different hardware, different datasets, and different numbers of concurrent jobs. There is no single “best” setup that can be read off from first principles. What works has to be found by experiment, one change at a time, measured carefully enough that the effect we see can be traced to the change we made. The methodology this thesis follows (Section 3) is therefore inherently empirical: profile, hypothesize, change *one* thing, measure, decide.

In short, the goal of this thesis is to make AI4Land training computationally more efficient on MareNostrum 5: to turn a pipeline that leaves its most expensive hardware idle into one that keeps that hardware busy. The point is not only to spend a finite allocation better. A pipeline that trains several times faster also shortens the loop between having

an idea and seeing its result, so the team can run more experiments in the same amount of time and iterate on the science more often. The next section turns this goal into a small set of specific, measurable objectives.

2 Objectives

The main objective of this thesis is to identify and resolve the dominant performance bottlenecks in the AI4Land training pipeline running on MareNostrum 5. The phrase *performance bottleneck* refers to the component of the pipeline whose throughput limits the throughput of the whole. By analogy with the neck of a bottle: removing the constriction in one place shifts the limiting role to the next-narrowest section, which then becomes the next target. A useful optimization effort is one in which the bottleneck moves several times, until the pipeline reaches a point where more work is no longer worth the expected gain. The concrete sub-objectives that organize the work are the following:

1. **Achieve the fastest possible training time** per epoch on a fixed compute budget, while preserving the predictive accuracy of the trained model. The primary quantitative measure used throughout the thesis is the per-batch wall-clock time in steady state, that is, the number of real seconds each batch takes once training has settled into its regular rhythm. We track it together with the full spread of those times over a window long enough to capture the tail as well as the median. The tail is what matters here. As the results show, the per-batch time is dominated by a small number of long stalls, so it is the tail of the distribution, not the median, that sets the epoch wall time.
2. **Make it possible to see more of the world within the same wall-clock budget.** For AI4Land it means putting more distinct samples through each epoch, so that the model sees more of the available place-and-year combinations in the same wall-clock time. The sampling scheme that gives this phrase its concrete meaning is described in Section 3.3. The two paths to it are higher throughput (objective 1) and higher hardware utilization, for example by increasing the effective batch size at constant epoch time.
3. **Quantify each optimization** through profiles produced with NVIDIA Nsight Systems and through per-batch timing logs collected outside any profiling window. A measured speedup must be tied to the specific component we changed, not to incidental factors such as the state of the operating system’s metadata cache or contention with other users on the cluster, both of which can vary substantially between otherwise identical runs.
4. **Preserve model accuracy.** Any change to the data pipeline (for example moving from multiple Zarr stores to a single store, or applying or skipping an offline preprocessing pass) must produce input tensors that are bitwise equivalent to the baseline. When bitwise equivalence cannot be guaranteed (for example because of floating-point summation order inside a normalization step), the training and validation losses must be statistically indistinguishable from those of the baseline over a comparable number of batches. A pipeline that is fast but degrades the science is not, in this context, an optimized pipeline.
5. **Produce reusable guidelines.** The optimizations identified should be documented in a form usable by AI4Land contributors, by the ELLIOT project that intends to reuse the pipeline, and by users of comparable scientific deep-learning workloads on HPC systems more broadly. A speedup that depends on one very specific hardware setup, or that breaks when the dataset size changes, is less valuable than a smaller speedup that holds across many setups.

3 Methodology

3.1 Literature Review

This is an applied optimization problem. There is no single theory that says, in advance, where a deep-learning data pipeline will be slow on a given machine; the answer depends on the model, the data layout, the storage hardware, and how many other jobs run at the same time, and it has to be measured. The useful prior work is practical, and it falls into four threads.

Tuning the data loader. Practitioner guides for PyTorch’s `DataLoader` agree on a short list of settings: run several worker processes, pin sample memory so it transfers to the GPU faster, keep the workers alive between epochs, and let each worker prepare a few batches ahead [24, 26, 33]. The AI4Land pipeline already did most of this. These settings hide the cost of reading a sample behind other work, but they do not make the read itself cheaper. One piece of advice does, and it is the one this thesis pursues: store the data in fewer, larger files.

Reading chunked data into training. A newer line of work feeds data straight from a Zarr store into the training loop, rather than first writing out one file per sample, and reports large speed-ups once the chunk shape matches how the data is actually read [11, 36]. These tools mostly target cloud storage or a direct path into the GPU. AI4Land instead reads from the shared filesystem of a supercomputer, where, as the next thread shows, the cost is dominated by the act of opening files.

The filesystem as the bottleneck. On supercomputers, the shared parallel filesystem is a known weak point for deep learning. It was built to stream a few very large files, while training asks for huge numbers of small, scattered reads. Reports from large machines show the filesystem’s bookkeeping, the lookups it performs to locate each file, saturating long before the data itself does [19, 39]. The common fixes add a caching layer or copy the data onto each compute node’s local disk [22], and the established practice for big scientific data is analysis-ready, cloud-optimized storage in sensibly sized chunks, rather than a dataset split into millions of tiny files [1, 10]. This thesis takes the portable route and simply cuts the number of files each sample opens, so no special hardware or caching layer is needed.

AI4Land and downscaling. The work here extends the AI4Land project directly. Its papers describe the U-Net, its inputs, and how training spreads across GPUs, reporting near-perfect scaling from four to thirty-two GPUs and about three hundred samples per second at the largest size [27, 28]. What they do not report is where the time goes inside a single GPU. Scaling well across many GPUs says nothing about whether each one is kept busy, and the profiles in Section 4 show that it was not. That is the gap this thesis fills.

3.2 Tooling

The optimization work in this thesis uses a small set of tools that the reader will encounter repeatedly. Several have been mentioned in passing in the previous subsections; the

descriptions below give each tool its own paragraph and define what it does in enough detail to make the rest of the document self-contained.

PyTorch [32]. PyTorch is an open-source deep-learning library originally developed by the AI research lab at Meta and now maintained by an independent foundation. It provides multidimensional arrays called *tensors* that can be moved transparently between CPU memory and GPU memory; automatic differentiation, meaning the ability to compute the gradient of any function expressed as a composition of PyTorch operations without the user writing a derivative by hand; a library of pre-built neural-network layers (convolutions, linear layers, normalization layers, activation functions); and an optimizer module that implements gradient-descent variants. AI4Land’s U-Net, its training loop, and its dataloader are all built on PyTorch.

Hugging Face Accelerate [18]. Accelerate is a wrapper library on top of PyTorch developed by Hugging Face, the company best known for hosting the open-source model and dataset hub of the same name. Accelerate abstracts away the boilerplate that a user would otherwise have to write to run a training loop on multiple GPUs or multiple nodes: it handles distributed initialization, gradient synchronization, mixed-precision arithmetic, gradient accumulation, and the launching of one process per GPU through the `accelerate launch` command. The AI4Land training entry point is launched through Accelerate; the details of the distributed-training scheme it sets up are described in the Current Training Pipeline subsection below.

Zarr [30, 42]. Zarr is a chunked, compressed, N -dimensional array storage format designed for scientific datasets that are too large to load into memory all at once. A Zarr store consists of a directory tree of small binary files, one per “chunk” of the underlying array, plus a small header file that records the array’s overall shape, data type, chunk layout, and compression scheme. The reference Python implementation opens a store (on local disk, on a parallel filesystem, or behind a cloud-bucket endpoint), and slicing the array triggers it to read only the chunk files that intersect the slice; everything else stays on disk. AI4Land’s training data is stored in this format.

xarray [20]. Xarray is a Python library that wraps NumPy-style and Zarr-style arrays with named dimensions and labelled coordinates, in the spirit of the netCDF file format that the geophysical community has used for decades. Where a raw NumPy array can only be indexed by integer position, an xarray Dataset can be indexed by the underlying physical coordinate, for example by year, latitude, and longitude. AI4Land’s data-loading code opens its Zarr stores through xarray and addresses slices by named coordinate rather than by integer index, which makes the code easier to read but adds a thin layer of bookkeeping on every access.

Hydra. Hydra is a configuration management library, also from Meta’s open-source ecosystem, that AI4Land uses to assemble a single training run’s configuration from layered YAML files. A YAML file is a plain-text file that lists settings as indented name-and-value pairs, meant to be easy for a person to read and edit. The configuration that Hydra carries sets the patch size, the batch size, the loss-function weights, the dataset path, and every other runtime parameter that the rest of this thesis will refer to.

SLURM [41]. SLURM, the Simple Linux Utility for Resource Management, is the workload manager that runs on MareNostrum 5 and on most other modern HPC clusters. The user submits a shell script through the `sbatch` command, decorated with comments that SLURM parses as resource requirements: number of nodes, GPUs per node, CPU cores per task, wall-clock time limit, partition name. SLURM queues the job until matching resources are free, then allocates them, launches the script on the allocated nodes (one process on the first node, with the rest reachable through SLURM’s `srun` command), and tears the allocation down when the script exits.

NVIDIA Nsight Systems [31]. Nsight Systems is the system-level profiler used throughout the optimization work in this thesis. A *profiler* is a tool that records, with timestamps, the events that occur during the execution of a program: function entries and exits, system calls, GPU kernel launches, memory transfers, blocking primitives. The profile is replayed visually in an interactive viewer, where each thread of every process, each GPU stream, and each system-call category appears as a horizontal track on a common time axis. Looking at the resulting timeline, the analyst can see how the components synchronize with each other and where any of them spends its time waiting on another. Nsight Systems is designed for workloads that involve both CPU activity and GPU activity, which is the case for any deep-learning training job.

NVTX. The NVIDIA Tools Extension (NVTX) is a lightweight C library, exposed in PyTorch through a small annotation interface, that lets the application annotate its own timeline with named ranges. When Nsight Systems captures a process that uses NVTX, the named ranges appear on the timeline as coloured bars, making it possible to identify application-specific phases at a glance rather than having to reverse-engineer them from the raw call stack. The AI4Land training loop is annotated with one range per phase of a training step (`data_loader`, `forward`, `backward`, `optimizer_step`) and one range per step named after its index (`step_10`, `step_11`, and so on); these are the labels that appear in the profiling results of Section 4.

uv [3]. `uv` is a Python package and project manager written in Rust by Astral Software, a company founded in 2022 that builds developer tools for the Python ecosystem. `uv` plays the same role that `Conda` or `pip-plus-virtualenv` have traditionally played: it installs Python packages from the public package index, resolves dependency versions to a mutually compatible set, builds isolated environments, and runs scripts inside those environments. The qualitative differences are speed (`uv`’s solver and downloader are roughly an order of magnitude faster than `Conda`’s), determinism (`uv` produces a lockfile that pins every transitive dependency to an exact version with hash digests), and interoperability (`uv` works on top of the standard `pyproject.toml` project description, which is the format the rest of the Python ecosystem also reads). AI4Land’s development environment was migrated to `uv` as part of this thesis.

ruff [2]. `ruff` is a Python linter and code formatter, written in Rust by the same Astral Software that builds `uv`. A *linter* is a tool that scans source code for stylistic problems and likely bugs (unused imports, undefined names, inconsistent indentation, suspicious patterns that often indicate mistakes); a *formatter* is a tool that automatically rewrites source code into a canonical style so that all developers contribute changes in a uniform

layout. ruff bundles both roles in a single binary, again roughly an order of magnitude faster than the historical combination of separate Python tools it replaces.

3.3 The Current Training Pipeline

Training the U-Net described above consists of repeatedly presenting examples to the model and adjusting its weights so that its predictions get closer to the corresponding ground-truth land-use map. A single example is called a *sample*, and consists of a fixed-size spatial patch of inputs (the multi-channel feature stack described earlier) paired with the same patch of ground-truth land-use classes drawn from HILDA+. AI4Land uses patches of 256×256 pixels, drawn from globally distributed locations chosen by a Farthest-Point-Sampling procedure that spreads them across the globe. Many samples are processed together in a *batch*, which is the natural unit of computation on the GPU. The loss function combines two cross-entropy terms with equal weight (0.5 each): an *overall* term computed over every valid pixel of the sample, and a *change-aware* term computed only over pixels whose land-use class differs between adjacent years. For each batch, the model produces predictions, computes the combined loss, back-propagates that loss through the network to compute the gradient with respect to every weight, and updates the weights by a small step in the direction that reduces the loss. A full pass through every sample in the training set, one batch at a time, is an epoch.

This sampling scheme is what gives the second objective of the thesis its concrete meaning. The full training set is every spatial patch at every year in the record, which is far too large to traverse in a single epoch. Each epoch therefore draws only a subset of it. The Farthest-Point-Sampling procedure spreads the chosen patches across the whole globe, so every epoch covers the world spatially, but for any one location only a few of its years are visited in a given epoch, and a different draw of years is used in the next one. One epoch might show the model the Barcelona region in 1983, a region of Zimbabwe in 1995, and the Washington area in 2010; the following epoch might show Barcelona in 1962, Zimbabwe in 1987, and Washington in 1974. Over many epochs the model sees a wide range of years at each place, but only a small slice of them per epoch. “Seeing more of the world” therefore means fitting more of these place-and-year samples into each epoch. Because the training years are fixed by the dataset, the lever is not a longer time span but a higher number of samples per epoch: a data path that delivers samples several times faster could let each epoch visit, say, two years per location instead of one at the same wall-clock cost, so the model sees more of the available combinations in the same time.

The hardware on which this training runs is MareNostrum 5 (MN5) [4], the pre-exascale supercomputer operated by BSC on behalf of the European High-Performance Computing Joint Undertaking (EuroHPC JU) [14]. A supercomputer is a tightly coupled cluster of hundreds or thousands of compute nodes connected by a high-speed interconnect, designed to run scientific workloads that do not fit on a single workstation. “High-Performance Computing” is the umbrella term for this class of machine and the workloads run on them. MareNostrum 5 has several partitions optimized for different workload classes; the one this thesis targets is the accelerated partition (MN5 ACC), which is built around NVIDIA H100 graphics processing units. A GPU is a many-core processor originally developed for real-time graphics rendering and now dominant in deep-learning workloads because its hardware architecture (thousands of arithmetic units operating in lock-step on vector operands) is well matched to the dense matrix multiplications at the core of

neural-network training and inference. The H100 is NVIDIA’s flagship data-centre GPU released in 2022 and the workhorse of large-scale AI training in 2024 and 2025. Each MN5 ACC node is equipped with four H100 GPUs, twenty CPU cores per task, and access to the shared scratch filesystem of the cluster.

The storage layer on which AI4Land’s training data resides is the General Parallel File System (GPFS), a distributed parallel filesystem that lets many compute nodes read and write the same files at the same time. GPFS keeps the contents of the files separate from their *metadata*, the bookkeeping that records each file’s name, size, permissions, and where its bytes physically sit on disk. The metadata is held on dedicated metadata servers. Before a compute node can read a file, it has to ask one of those servers to look the file up and report where its bytes are. The time spent on that exchange, travelling to the metadata server, locating the file, and getting the answer back, is what this thesis calls the *metadata latency*, or just the latency for short. A single lookup costs on the order of hundreds of microseconds when nothing else is competing for the servers, and climbs to several milliseconds when many jobs read at once and crowd them. Every file the pipeline opens pays this latency once before any of its bytes arrive. The Results section shows that this per-file latency, not the volume of data moved, is what governs the cost of loading a training sample.

The training data is stored on the GPFS scratch volume in Zarr format, introduced in the Tooling subsection above. The AI4Land project uses the term *Analysis-Ready Cloud-Optimized* (ARCO) to refer to a single Zarr store that fuses all the relevant variables of a dataset on a common spatial and temporal grid, ready to be consumed by a training loop without further preprocessing. Section 4 compares an ARCO single-store layout against the original five-store layout that the AI4Land team inherited.

The samples are loaded into the model by a PyTorch [32] **DataLoader**. PyTorch is the deep-learning framework used throughout AI4Land; its **DataLoader** component turns a **Dataset** (a Python object that knows how to produce one sample given an integer index) into an iterable stream of batches. The **DataLoader** runs a configurable number of *worker processes*, each of which fetches samples from disk in parallel with the main training loop running on the GPU. Each worker also *prefetches* ahead: it does not wait to be asked for the next sample, but rather starts producing samples immediately and queues finished samples up to a configurable depth. Two parameters govern this behaviour. The first is the number of worker processes the loader spawns. The second is the prefetch factor: how many samples each worker keeps queued ahead of the main loop. Together they set the largest number of samples that can be “in flight” between disk and the GPU at any time.

To use all four GPUs in a node simultaneously, AI4Land trains under PyTorch’s Distributed Data Parallel (DDP) mechanism. DDP replicates the U-Net on every GPU and partitions the batches across them: each GPU processes a different slice of the batch, computes a local gradient, and the local gradients are then averaged across GPUs by a collective operation called an *all-reduce* before the optimizer step that updates the weights. The result is that all four GPUs in the node produce a single, consistent set of weight updates, as if the batch had been processed on a single, four-times-larger GPU. Each of these per-GPU worker processes is called a *rank*; throughout this thesis one rank corresponds to exactly one H100 GPU, so a “four-rank” run uses four GPUs and an “eight-rank” run uses eight GPUs across two nodes. The orchestration of DDP, gradient accumulation, and mixed-precision arithmetic is handled by Hugging Face Accelerate [18], the wrapper

library introduced in the Tooling subsection above. The specific mixed-precision format used by AI4Land is *bfloat16* (the `bf16` option of Accelerate), a sixteen-bit floating-point format defined by Google and adopted by NVIDIA’s H100 hardware that preserves the dynamic range of the standard 32-bit float (the same eight-bit exponent) at the cost of trading away precision in the fraction; the practical effect is that the forward and backward passes of the U-Net consume roughly half the GPU memory and run noticeably faster than they would in 32-bit, while remaining numerically stable for the loss values involved.

Jobs are submitted to MN5 through the Simple Linux Utility for Resource Management (SLURM) [41], the workload manager introduced in the Tooling subsection above. A representative single-node AI4Land training configuration uses one task per node on the ACC partition, where each node provides 80 CPU cores and four H100 GPUs to that task. Accelerate then spawns one Python process per GPU (so four per node), each effectively using a quarter of the node’s CPU budget, and each process runs its own dataloader with twelve worker processes (forty-eight workers per node in aggregate). The dataloader is configured with a batch size of eight (eight samples processed at a time per GPU) and a prefetch factor of four. Memory pinning is enabled, which speeds up the copy from host memory to the GPU by holding each sample in a fixed region of host memory that the operating system will not move. With gradient accumulation over four optimizer-step intervals, this yields an effective batch size of $4 \times 8 = 32$ samples per weight update on a single GPU and $4 \times 32 = 128$ samples per weight update across the four GPUs of a node. Under this configuration, the EuroHPC submission of the AI4Land team reports a single epoch of the historical model in approximately two hours and forty minutes on one node [28]. The same paper reports near-linear scaling from one to eight MN5 ACC nodes (four to thirty-two H100 GPUs), with a parallel efficiency of 97.7% at thirty-two GPUs and a sustained throughput of approximately three hundred samples per second at peak.

3.4 The U-Net Architecture

The U-Net used by AI4Land follows the original encoder–decoder design of Ronneberger et al. [34], scaled to the inputs and outputs of the downscaling task. The reference implementation lives in the AI4Land Python package.

The encoder consists of five stages of progressively reduced spatial resolution. The first stage applies a *double convolution*, that is, two consecutive 3×3 convolutions each followed by batch normalization and a Rectified Linear Unit (ReLU) activation, which lifts the multi-channel input into 64 feature channels at the input resolution. The next four stages each halve the spatial resolution through a 2×2 max-pooling and double the channel count, producing feature maps with 128, 256, 512, and 1024 channels at one-half, one-quarter, one-eighth, and one-sixteenth of the input side length, respectively. The decoder mirrors the encoder: each of its four stages upsamples its input by a factor of two (bilinear interpolation with corner alignment), concatenates the result with the corresponding encoder feature map at the same resolution (the U-Net *skip connection*), and runs another double convolution. After the four decoder stages the network has returned to the input resolution with 64 feature channels. A final 1×1 convolution maps those 64 channels to one logit per land-use class, producing an output map of class scores at the input resolution. Across the entire encoder–decoder, all internal convolutions use 3×3 kernels with padding one (so spatial dimensions are preserved within a stage), and the

only spatial reduction is the max-pooling between stages.

Two of the inputs are categorical rather than continuous: the Köppen–Geiger climate zone and the autoregressive HILDA+ prior. Both are turned into dense feature maps through learned embedding layers (the same mechanism that turns word tokens into vectors in natural-language models). The Köppen–Geiger embedding has dimension five (the 31 zone classes plus a padding row); the HILDA+ embedding has dimension two (the 8 land-use classes plus a padding row that the masking strategy uses). The continuous LUH2 channels, the static geophysical priors, and the two categorical embeddings are concatenated along the channel axis before being fed into the first double convolution.

For multi-year prediction, the U-Net is wrapped in a recurrent rollout module that unrolls the same network autoregressively. At year t , the model’s own prediction at $t - 1$ is reused as the HILDA+ prior for the next forward pass; the same weights are applied at every step (the network is recurrent in time even though, as a U-Net, it is feed-forward in space). During training a *teacher forcing* schedule linearly decays the probability of using the ground-truth HILDA+ map (rather than the model’s own previous prediction) from one to zero over the first 5 000 optimizer steps, which gradually exposes the network to its own errors without destabilizing the early phase of training.

3.5 Optimization Loop and Measurement

Optimizing a pipeline like this is empirical work, closer to art than to science. As one bottleneck is removed the next one appears, and the settings interact, so the best setup cannot be worked out on paper. The method is therefore a loop. We profile a real training run to see where the time goes, make one concrete guess about the slowest part, change a single thing to test that guess, and compare the result against the unchanged baseline. A guess that turns out wrong is still useful, because it rules a cause out. The plan was to fix the data loader first and then turn to the GPU. So the loop runs first against the data side, until the data loader can feed samples as fast as the GPU trains on them. Past that point the data is no longer the limit and the GPU sets the pace; the same loop then carries on against the compute side, which is where the work continues beyond this thesis.

This staged way of working has a precedent in Earth System modelling. A performance-analysis framework from the Barcelona Supercomputing Center follows the same path on climate models: measure how fast a model can run, find what stops it scaling, then study why [38]. That work also states the case behind this thesis, that cheap analysis at small scale points to fixes which pay off in the large runs, and that time saved on computation is returned to the science as more experiments, finer resolution, or larger ensembles.

Two habits keep each comparison sound. We change one thing at a time, so any difference in timing has a single possible cause. And before timing any change to the data layout, we check that it still hands the model exactly the same numbers as before, so that a “faster” pipeline is never just one that quietly feeds the model different data.

The harder problem is that MareNostrum 5 is a shared machine. Hundreds of other jobs read and write the same filesystem at the same time, and how quickly a file is read also depends on whether it was read recently and is still held in memory. Both of these change from one run to the next, and neither is under our control. A single timing can therefore mislead: the same setup can look fast on a quiet filesystem and slow a minute later when a neighbour floods it. To guard against this, every configuration is measured several times

rather than once. We discard the fastest and the slowest reading of each set and average the rest, so that one unlucky spike or one lucky fast read does not decide the reported number.

4 Results

4.1 Establishing a Reproducible Workflow

When you optimize a pipeline, you need to iterate quickly on both the source code and the project dependencies. Changes must not disturb the work of colleagues, who may need a different set of dependencies. And any experiment must be reproducible on another machine at a later date. The AI4Land workflow, as inherited, did not offer these properties. The first task of the optimization work was therefore to improve the code and the surrounding tools so that it did.

4.1.1 Conda limitations and the move to uv

The AI4Land project’s environment was managed by Conda. Conda is a Python package and environment manager originally developed for the scientific computing community. It builds isolated environments, each with its own pinned Python and packages, and resolves dependencies through a satisfiability solver that searches for a mutually compatible set of versions across the user-requested packages and their transitive dependencies. Unlike pip, Conda also installs non-Python system libraries: the linear-algebra backend that NumPy links against, or the Compute Unified Device Architecture (CUDA) toolkit that PyTorch builds against. That made it well suited, historically, to scientific work where Python wrappers and native libraries had to match. Since uv became stable in 2024, many scientific teams have moved away from Conda for projects of AI4Land’s shape; the migration described here follows that broader trend.

Three problems stood in the way of the workflow the optimization work needed.

The first problem is storage. Conda environments are stored on disk as a full copy of every installed package, with no sharing of files between environments. A modern PyTorch-with-CUDA environment is already several gigabytes, and the AI4Land environment, with its scientific and geospatial dependencies layered on top, is correspondingly large. On a shared filesystem with per-user quotas, holding one environment per development branch was not feasible. The team had therefore settled on a single environment shared across all branches. The two further problems below are direct consequences of that decision.

The second problem is that, because the environment was shared, any change to a dependency on one branch became active for everyone on the same login node. Rolling back to reproduce an earlier result was not a self-contained operation: the change had to be undone globally and then redone after the reproduction. On top of that, because Conda installs system libraries alongside Python packages, the same native library can be brought in by several Conda packages at different versions, and which version ends up active is decided by the solver’s resolution order rather than by an explicit choice in the project’s configuration.

The third problem is that Conda’s solver itself is slow. A clean AI4Land environment takes three to ten minutes to build on the MareNostrum 5 login nodes. Every time a developer needed a different dependency set, they paid that rebuild cost before they could run an experiment.

uv [3] is the Python package and project manager introduced in the Tooling subsection of the Methodology. Its design addresses each of the three problems above.

The first piece is the shared package cache. `uv` stores every downloaded package once on disk, in a content-addressed cache, and hard-links from each environment back into that cache. When two environments use the same package at the same version, the underlying files are shared instead of duplicated. The disk cost of a per-branch environment is therefore the cost of the differences from the cache, not the cost of a full copy. Each developer can keep one environment per branch instead of sharing a single global one. The second piece is the lockfile. `uv` writes out a lockfile that pins every direct and transitive dependency to an exact version, together with a cryptographic hash. The lockfile is committed into the project; any other developer, or the same developer on a different machine, reads that file and builds the same environment in the same way. The third piece is speed. `uv`'s solver and downloader are written in Rust and run in parallel at every step where parallelism is available; a clean environment that took three to ten minutes under Conda now takes a few seconds under `uv`.

The migration itself was carried out in three steps. The first step was to write a `pyproject.toml` file, the standard project-description file adopted by the Python community. The file lists the project's direct dependencies and the version range accepted for each. The new file pins the project's runtime dependencies, including the CUDA-matched PyTorch build that the H100 partition requires. The second step was to reorganize the source tree into a packaged layout, with every AI4Land module under a single source directory. That turns the codebase into an installable Python package, which means it can be installed once on a node and used from any directory under SLURM rather than being executed by relative path from the checkout. The third step was to rewrite the entry-point scripts so that they import from the installed package rather than from neighbouring files.

After the migration, the three problems disappeared together. Clean environments build in seconds. The lockfile pins every dependency, so the same environment can be rebuilt on a different machine identically. Each branch can keep its own environment cheaply, so a code change on one branch no longer interferes with another branch's environment.

4.1.2 Code style and tests

`ruff` [2] is the Python linter and code formatter introduced in the Tooling subsection. The reason to adopt it is to keep code review focused on what actually changed and improve code readability by standardizing the style.

At the granularity at which the optimization work in this thesis operates, where a branch usually differs from the baseline by a single change, a diff that mixes the substantive change with unrelated reformatting is harder to read and slower to review. A diff that contains only the substantive change can be read at a glance.

Having a similar style across the codebase makes it much easier to understand it as a whole and pinpoint possible bugs. The reader learns what certain constructs look like and what they do, making it easier to recognize them in a different context. A consistent style also makes it easier to spot outliers, which are often bugs.

`ruff` is enforced through a *pre-commit hook*. A pre-commit hook is a small script that runs automatically before every recorded change to the codebase, and that blocks the change if the formatter or the linter reports an issue. The hook ensures that the project's style is the only one that is ever committed. Running `ruff` on the entire AI4Land codebase takes well under a second, so the hook does not slow developers down.

The last piece of the workflow change is two small test scripts that run inside SLURM and act as gates before any change is merged.

The first is a *smoke test*. The smoke test runs a short training epoch end to end on one H100 GPU and checks that the loss decreases and that no exception is raised. It catches gross regressions: a refactor that breaks the dataloader, a configuration change that silently disables an input source, a tensor-shape mismatch introduced by a layout change. The smoke test finishes in under five minutes.

The second is a *parity test*. The parity test draws a fixed set of samples from two pipeline variants and checks that the resulting input tensors are bit-identical (or, when floating-point summation order makes bit-equality impossible, that all per-sample statistics agree within a strict numerical tolerance). The parity test runs before any performance comparison between two layouts. A layout change that produces a different tensor is not a valid performance comparison, because the model would then be training on different data. Every layout change in this thesis was admitted only after the parity test passed.

4.1.3 Scientific and workflow impact

Together, these changes give the rest of the chapter what it needs. Branches build in seconds and do not interfere with each other, so experiments can be iterated quickly. The lockfile and the parity tests together pin both the software state and the data state of any reported measurement, so a result reported in this thesis can be reproduced on a different machine and at a later date. Code review focuses on the change being reviewed, because formatting noise is filtered out by ruff at commit time.

The payoff is not only practical but scientific. A pipeline that anyone can rebuild from a pinned lockfile and install as a package, rather than one that runs only inside a single hand-maintained environment, is far more likely to be picked up and reused by other groups. The literature on reproducible and reusable research software makes this case directly: software that others can install, rerun, and build on is more readily reused, and reproducibility is widely held to be a precondition for others to trust and extend a computational result [35, 5]. The migration described here is therefore also an investment in the reuse of the AI4Land pipeline, by the ELLIOT project and by any other group that wants to train on the same data.

4.2 Initial Profiling

Before changing anything in the pipeline, we needed to know where the time was actually going. The first measurement was therefore a profiling run against the AI4Land pipeline as the team had been running it, with no changes other than the instrumentation needed to record what each component was doing and when.

The instrumentation has two layers. The lower layer is NVIDIA Nsight Systems, the system profiler introduced in the Tooling subsection. Nsight Systems records, with timestamps, every function entry and exit, every system call, every GPU kernel launch, every memory transfer, and every operation in which one component of the program waits on another. The recorded events are replayed in an interactive viewer as a visual timeline, with one track per process, per GPU stream, and per system-call category. The upper layer is NVTX, the annotation interface also introduced earlier. The training loop is wrapped with

named NVTX ranges (`dataloader`, `forward`, `backward`, `optimizer_step`), and each individual training step is also wrapped in a range named after its index (`step_10`, `step_11`, and so on). On the Nsight timeline these ranges appear as coloured bars, which lets us identify at a glance both the high-level phase (which part of the training loop is running) and the exact step it belongs to, instead of having to read the raw call stack.

The profiling target was the production training loop on the multi-zarr layout, the data arrangement that the AI4Land team had been training on. In that layout each input source lives in its own Zarr store, so the dataloader fetches each sample by reading from five separate stores in turn. A representative Nsight Systems trace from this configuration is shown in Figure 2. At the very top of the trace is the GPU activity track. It is essentially idle through the whole window, with brief activity only at the left and right edges, where the forward, backward, and optimizer-step phases of the neighbouring training steps fall inside the recorded window. The middle of the trace is occupied by the NVTX ranges that name what the program is doing. The range that dominates the window is the `dataloader` phase of `step_11`, which alone takes about seventy-six seconds; the labels for the neighbouring steps are pushed to the extreme left and right of the window and are mostly hidden. At the bottom of the trace are the dataloader worker processes, each spending most of its recorded time blocked in `sem_wait`, the system call that suspends a thread until a semaphore is signalled. The pipeline is bound by data preparation on the CPU, and the most expensive resource in the system, the H100 GPU, sits idle while the dataloader workers do their work.

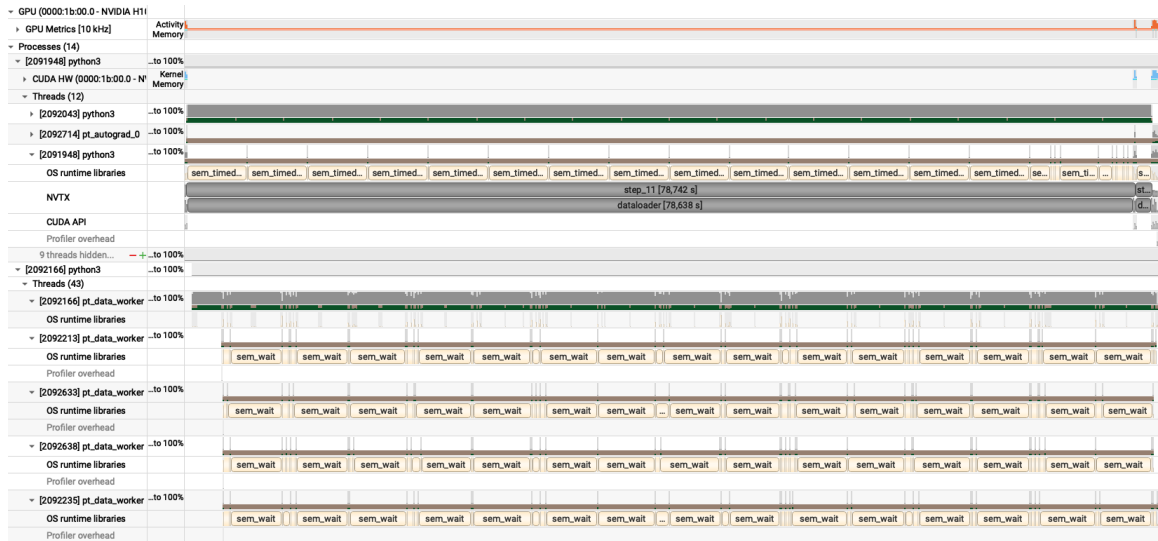


Figure 2: Representative Nsight Systems trace on the multi-zarr layout, single rank. Top: the GPU activity track, idle through most of the window and lit only at the extremes, where the forward, backward, and optimizer-step phases of the neighbouring steps fall inside the recorded window. Middle: the NVTX ranges; the `dataloader` phase of `step_11` dominates the window at about seventy-six seconds, with the labels of the neighbouring steps mostly hidden at the edges. Bottom: the dataloader worker processes, each spending most of its time blocked in `sem_wait`.

On top of the Nsight trace, the training loop also writes a wall-clock log of how long each batch takes. Reading that log shows a clear periodic pattern. About twelve consecutive batches finish in well under a second each, then one batch takes around eighty seconds,

then twelve more fast batches, then another slow batch, and so on. The period of twelve is not a coincidence: the dataloader runs twelve worker processes, and they never get the chance to build up the queue of prefetched samples the configuration asks for. The twelve workers each deliver one sample, the GPU drains those twelve in about a second, and the workers have not finished fetching the next round before the queue runs dry and the GPU is asking for more. The slow batch is the main process blocked while all twelve workers open files and refill from disk at once; the fast batches are those twelve already-prepared samples being consumed. The configured prefetch cannot hide the stall, because one refill takes far longer than the GPU needs to burn through everything the workers can stage ahead of it. The cadence is tied to the number of workers: with eight workers the slow batch would fall on every eighth batch and with twenty on every twentieth, so the worker count sets how often the stall is paid, a relationship the cost estimates later in this chapter rely on.

The Nsight trace has a third observation that points at where to look next. Each dataloader worker thread spends almost all of its recorded time blocked in a low-level operation that the operating system reports as a *kernel wait*, a state in which the thread is suspended by the kernel until something releases it. The kernel-wait signature is the same in two very different situations. It can mean “the worker is idle, waiting for the main process to ask for the next sample”. It can also mean “the worker is busy inside its per-sample fetch but blocked inside a library call that itself uses a kernel wait at a lower level”. The Nsight trace on its own cannot say which of the two is happening, because it records that a worker thread was blocked but not which call inside the worker caused the block. The trace therefore frames the next question rather than answering it. Either the inter-process queue between the workers and the main process is the limiter, or some library inside the per-sample fetch is.

One quantity sets the scale for everything that follows. In this window the pipeline feeds the GPU far more slowly than the hardware allows: the bytes each sample needs are small, yet they arrive roughly two hundred times slower than the filesystem can stream data, a gap measured directly later in this chapter (Section 4.6.3). A shortfall that large cannot be blamed on the volume of data. It points instead at a fixed cost paid once per read, however few bytes that read returns, so the lever that matters is how many separate files a sample has to open. That number is inflated by the way the data is stored: the inputs are split across five separate Zarr stores, and every sample opens all five. The first and simplest step is therefore structural: collapse those five stores into one, so each worker opens one store per refill instead of five. The change leaves the libraries inside the per-sample fetch untouched and gives a cleaner baseline against which library-level hypotheses can be tested in the subsections after it.

4.3 From Multiple Stores to a Single Store

The AI4Land team had been training on the *multi-zarr* layout, in which the training inputs are split across five separate Zarr stores, one per input source. The five sources are the LUH2 dynamic forcings, the HILDA+ target, the static geophysical priors (elevation, slope, aspect, soil composition), the Köppen–Geiger climate zones, and the ISIMIP2b population. Every worker, on every sample, opens all five stores, reads the relevant spatial and temporal slice from each, and assembles the per-sample input tensor by aligning the five slices on a common grid.

The hypothesis that motivates collapsing the five stores into one is plain. Opening five stores per worker reads five sets of headers, traverses five directory trees, and acquires five sets of file handles, whereas opening a single combined store does that work once. Bundling AI4Land’s variables into a single analysis-ready store on a common spatial and temporal grid should remove the per-sample fixed cost of the four extra stores and produce a more coherent access pattern on the parallel filesystem. The change is structural, not algorithmic: the model and the training code do not have to change. It was the first lever we tried, and the simple expectation behind it is that fewer stores mean less per-sample work. As the measurements below show, bundling does help, and substantially. But the same runs also turn up a result the simple expectation does not predict and cannot explain, an early sign that the number of stores is not the whole story. That thread is picked up once the bundling numbers are in, and runs through the rest of the chapter.

The bundling was implemented by writing all five input sources out into a single new Zarr store. Two variants are used throughout the rest of this chapter (Table 1), both covering a shorter span than the multi-zarr layout. The first covers the historical window 1960–2015 at the full resolution and is 213 gigabytes on disk. The second covers a shorter window, 2000–2015, and is 66 gigabytes; the smaller variant exists to make per-sample behaviour easier to interpret in isolation. Both single-zarr variants serve bit-identical input tensors to the dataloader (verified through the parity test described in the previous subsection).

Layout	Size on disk	Chunk files	Time window
Multi-zarr (five separate stores)	680 GB	6,732,258	1850–2020
Single-zarr, full window	213 GB	2,448,648	1960–2015
Single-zarr, short window	66 GB	710,568	2000–2015

Table 1: The three layouts used in the rest of this chapter, with the number of separate chunk files each one stores. The multi-zarr total is dominated by the LUH2 store, which alone holds 606 gigabytes and about 5.8 million of the 6.73 million chunk files; the four other multi-zarr stores add about 74 gigabytes. The two single-zarr variants hold the same kind of data and differ only in how many years they cover, and so in size and file count: the full window holds about three and a half times as many chunk files as the short window. File counts were obtained by walking the stores directly, except for the full single-zarr store, whose count is derived from the store geometry and matches the measured short store to the file.

The comparison was performed under the production training loop with identical model hyperparameters and identical SLURM configuration (a single rank, twelve workers, batch size eight, gradient accumulation four). Per-batch wall-clock logs of fifty consecutive batches were collected for each layout. The observation is that the periodic slow batch survives in both layouts, but its magnitude is very different (Figure 3). The multi-zarr layout produces slow batches around eighty seconds every twelve batches; the single-zarr layout produces slow batches around ten seconds at the same cadence. The cadence is unchanged, because it is set by the same twelve-worker refill cycle as before. The magnitude shrinks by roughly a factor of eight. What drives a gap that size, given that the single-zarr store also holds far fewer chunk files than the multi-zarr one (Table 1), is the thread the rest of the chapter follows.

This was a clear improvement, and the single-zarr layout became the new baseline for every subsequent measurement. We would have moved on at this point if a second, unexpected

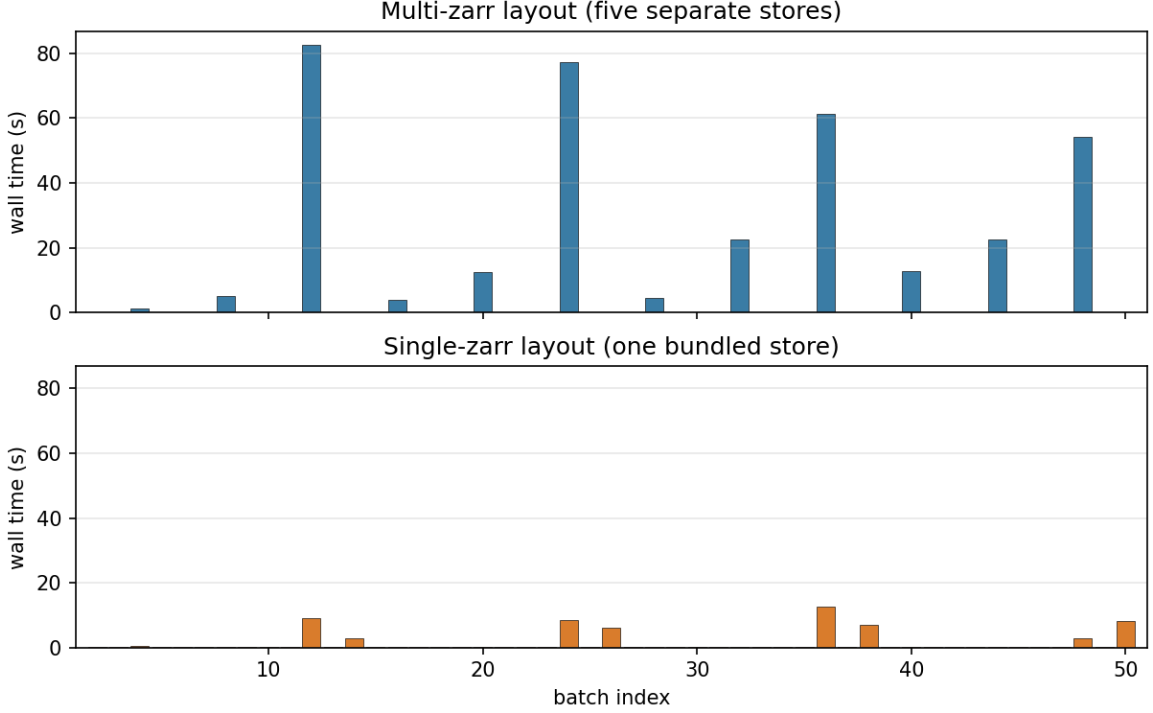


Figure 3: Per-batch wall-clock time during the first fifty batches of a training run, on the multi-zarr layout (top) and on the single-zarr layout (bottom), under the same model and SLURM configuration. The horizontal axis is the batch index; the vertical axis is the wall-clock time for that batch, on a shared scale. Both layouts show the same periodic pattern of fast batches with one slow batch every twelve. The slow batches last around eighty seconds on the multi-zarr layout and around ten seconds on the single-zarr layout.

observation had not appeared in the same set of runs. The 213-gigabyte single-zarr store had slow batches about three times larger than the 66-gigabyte single-zarr store, even though their chunk layout, patch size, input-source count, and worker configuration were identical. The two stores differ only in their total on-disk size, which is itself a function of how many years they cover. The size dependence was reproducible across independent runs (Table 2).

Reproduction	66 GB store	213 GB store	Ratio
First	8.0 s	28.0 s	3.5×
Second	10.3 s	31.1 s	3.0×

Table 2: Mean magnitude of the slow batch in two independent reproductions, on the same single-zarr layout, the same chunk layout, and the same access pattern; only the total on-disk size changes. The 213-gigabyte store has slow batches about three times larger than the 66-gigabyte store.

The size effect is unexpected, because the dataloader never reads the whole store. On each sample it reads a small spatial slice at a few timesteps, so the bytes transferred per sample do not depend on how many other slices sit on disk. Yet the per-sample wall time does. This is the second open question the bundling experiment raised. It is returned to in Section 4.6.6, once the diagnostic tools are in place: what is the per-sample cost actually made of, and why does it grow with the size of a store the fetch barely touches?

4.4 Removing the Synchronizer

With the single-zarr layout established as the new baseline, the next thing to check was whether the residual kernel waits inside each worker’s fetch were caused by a specific library or by the inter-process queue between workers and the main process. The most natural library to suspect first was Zarr itself. The reference Python implementation of Zarr supports concurrent access to a store across multiple processes through a *process synchronizer*. The process synchronizer is a small object that holds a file-based lock and that serialises operations across processes through it. Acquiring such a lock goes through the standard Unix file-locking interface, which uses a kernel wait at the lowest level, and therefore shows up in profiler traces with the same signature as the worker stalls we were trying to explain. The synchronizer was enabled on every Zarr store in the AI4Land code, as a precaution against multiple workers writing concurrently and corrupting the data. A reasonable first move was therefore to disable it and measure again.

Three independent comparisons were carried out on the single-zarr layout (Table 3). Inside the production training loop, the per-batch wall-clock pattern is the same with the synchronizer enabled and disabled: same fast-batch baseline, same slow batches at the same cadence. In a standalone, single-thread per-sample fetch (the standalone profiler is described in the next subsection), the mean per-sample time is essentially identical with the synchronizer enabled and with it disabled, within run-to-run noise. Inside the Nsight trace, the share of time attributed to kernel waits is essentially the same in the two cases.

Measurement	Synchronizer on	Synchronizer off
In-loop per-batch pattern	identical	identical
Standalone single-thread fetch (ms)	1177	1178
Kernel-wait share of Nsight trace	89.2 %	82.9 %

Table 3: Same single-zarr layout, synchronizer on versus off, three independent observations. The in-loop per-batch pattern is identical between the two configurations. The standalone single-thread per-sample fetch time is identical within noise. The share of the Nsight Systems trace attributed to kernel waits is essentially the same.

Reading the source of the Zarr library closes the question. In the version of Zarr in use, the synchronizer’s lock is only acquired on the chunk-write path. The chunk-read path, which is the only path the dataloader exercises, never acquires the lock and therefore never contends with anything that does. On a read-only workload the synchronizer is simply not engaged, regardless of whether it is enabled in the configuration.

Two consequences follow. The configuration switch that toggled the synchronizer was removed from the AI4Land code, since it had no effect on the read path. The synchronizer instance itself was kept in place defensively, in case a concurrent writer is ever introduced. More importantly for the rest of the chapter, the kernel-wait pattern is real but its cause is not the synchronizer. It has to come from somewhere else: from the inter-process queue used by the PyTorch DataLoader, from the parallel filesystem’s client daemon, or from some library called inside the per-sample fetch. The investigation continues.

4.5 Isolating the Dataloader

The measurements collected so far were taken with the full training loop active, so every per-batch number reflects both the dataloader’s work and the model’s forward and backward passes. Attributing a measured speed-up to the dataloader, rather than to incidental fluctuations on the model side, requires isolating the dataloader.

To do that, a standalone dataloader profiler was developed. The profiler runs the AI4Land dataset code without instantiating the model or the optimiser; it executes a controlled sequence of per-sample fetches and records timestamps at every interesting boundary. Each measurement runs inside a fresh subprocess, so the child cannot inherit any configuration state that might bias the timing. The profiler is organised into three phases that progressively widen the measurement scope.

Phase 1: time spent in each input source. Fifty samples are fetched in sequence with no worker processes and no concurrency. Inside each fetch, the time spent reading each input source is recorded separately. The total per-sample fetch in this configuration is approximately 1.25 seconds. The breakdown across sources is heavily uneven (Table 4). A single LUH2 read takes around 428 milliseconds on average, and because every sample reads LUH2 at two timesteps it accounts for about 68 per cent of the per-sample fetch. The other sources together account for less than a third of the time. The output tensor that arrives at the model is 9.5 megabytes per sample, so the effective throughput of one single-threaded reader is around 8 megabytes per second, far below the parallel filesystem’s single-stream bandwidth (revisited in detail in Section 4.6.3).

Input source	Reads per sample	ms per read	ms per sample	Share
LUH2	2	427.9	855.8	68.0 %
Static	1	105.1	105.1	8.4 %
Population	2	55.3	110.6	8.8 %
HILDA+ target	1	48.9	48.9	3.9 %
HILDA+ prior	1	47.8	47.8	3.8 %
Köppen–Geiger	2	43.3	86.6	6.9 %
Total			1254.8	100 %

Table 4: Mean time spent in each input source during a single-thread per-sample fetch on the single-zarr layout, averaged over fifty samples. The LUH2 read is fetched at two timesteps (the current year and the previous year, the latter feeding the autoregressive prior) and dominates the per-sample budget.

Phase 2: load step versus preprocess step. Each per-source call is split into a *load* step that issues the actual read against the Zarr store, and a *preprocess* step that performs the in-memory work on the loaded bytes (filling missing values, normalising to a common scale, stacking the variables, transposing axes, casting to the data type the model expects). The split shows that the load step accounts for approximately 92 per cent of the per-sample fetch and the preprocess step for approximately 8 per cent. This caps any optimization aimed at the preprocess step at 8 per cent of the per-sample cost; the load step is where the time is.

At this point the natural follow-up was whether the preprocess cost could be moved offline. Concretely, the team had been considering writing out a second copy of the dataset with the fill-NaN and normalize steps already applied to the on-disk bytes, so that the dataloader would not have to perform them at read time. The hope was that this would eliminate the dataloader bottleneck. Phase 2 made it possible to check the hope before the dataset was ever rewritten: a preprocessed variant of the same store was measured against the non-preprocessed variant under otherwise identical conditions, and the two returned indistinguishable per-sample fetch times and indistinguishable end-to-end throughput. The preprocess work that the bake-in would have removed (fill-NaN and normalize) was already only a small fraction of the cost. The residual preprocess time that remains in both layouts is the in-memory stacking, transposing, and reshaping, which an offline bake-in cannot remove. The preprocessed-store idea was therefore not the answer to the dataloader bottleneck and was not pursued further.

Phase 3: worker-count sweep. The dataloader worker count is swept through one, four, and twelve, with eighty batches run per configuration. Table 5 reports the resulting steady-state throughput.

Workers	Batch mean (ms)	Throughput (samples/s)
1	9325	0.9
4	2428	3.3
12	832	9.6

Table 5: Steady-state throughput on the standalone profiler under the single-zarr layout, eighty batches per configuration. Throughput scales at about 0.8 samples per second per worker. The twelve-worker throughput equals the worker count divided by the single-thread per-sample fetch time, which means the dataloader is bottlenecked on per-sample work rather than on queue mechanics.

Throughput scales approximately linearly with worker count, at roughly 0.8 samples per second per worker. The twelve-worker throughput, 9.6 samples per second, equals the worker count divided by the single-thread per-sample fetch time: 12 divided by 1.25 seconds. The linear scaling, and the exact identity, confirm that the dataloader is bottlenecked on per-sample work and not on the inter-process queue or any other coordination layer. The lever is per-sample fetch time, not worker count.

Three of the candidate causes from the initial profiling are ruled out by these three phases. The kernel-wait pattern is not idle queue waiting, because if the queue were the limit then adding workers would not help; Phase 3 shows that adding workers does help, linearly. The cost is not in the preprocess step, because Phase 2 caps that contribution at 8 per cent and the preprocessed-store check above reproduces the same numbers. The cost is concentrated in the load step of each input source, with LUH2 being the largest. The next subsection drills into what a single chunk read on that path is actually doing.

4.6 Diagnosing the Bottleneck on Synthetic Data

4.6.1 Anatomy of a single chunk read

A separate diagnostic times the layers of a single chunk read. Each chunk fetch is wrapped in nested timers: the bytes-from-disk layer (the read against the parallel filesystem, which

covers both the metadata latency to locate the file and the transfer of its bytes), the Zarr index lookup (which resolves a logical chunk index to a file path), the decompression layer, and the xarray slice-and-materialise layer (which turns the raw bytes into the labelled in-memory array the model consumes). Two independent measurements on production chunks agree closely (Table 6).

Layer of a single chunk read	Cold cost (ms)
Bytes from disk (metadata latency plus transfer)	19–20
xarray slice-and-materialise	16
Decompression	0.3
Zarr index lookup	~0

Table 6: Cost of one cold chunk read on a production chunk, two independent measurements that agreed to within a millisecond per layer. The filesystem read and the xarray step are the two substantial costs and are of comparable size; decompression and the index lookup are negligible.

Two layers carry essentially all the cost, and they are of comparable size. The read against the filesystem is about 19 to 20 milliseconds per chunk, and that figure barely moves with the compressed size of the chunk: a 25-kilobyte chunk and a 122-kilobyte chunk both take about the same time. A cost that is almost flat in the number of bytes but is paid once per file is the signature of metadata latency, the time the filesystem takes to locate a file before any of its bytes can be read, rather than of byte transfer. The second substantial cost is the xarray step that turns the raw bytes into the labelled array, at about 16 milliseconds per chunk, almost as much as the read itself. Decompression, by contrast, is negligible: 0.3 milliseconds per chunk, an effective rate of about three gigabytes per second, so the codec and its compression level are not where the time goes.

We did not investigate the xarray figure further. It is pure Python work, so it is held back by the Global Interpreter Lock (GIL), the kernel-level mutex inside the Python interpreter that lets only one thread run Python bytecode at a time, and it cannot simply be threaded away. But how it scales (whether it is genuinely per-chunk or carries a fixed per-call minimum) and how much of the per-sample fetch it ultimately accounts for are questions this single measurement cannot settle; the xarray layer’s impact deserves a dedicated investigation. The merged store later puts a number on it, where this layer becomes the dominant residual cost (Table 9). What the per-chunk picture does establish cleanly is the read side: each chunk file carries its own metadata latency, and that cost scales with the number of chunk files a sample touches, not with their bytes. The next subsection counts those files.

4.6.2 Per-sample bytes and output tensor size

A second accounting, of how many files a sample opens, completes the diagnosis. Two numbers combine. The first is the number of read calls the dataloader issues per sample: 41, one for each variable at each timestep the sample needs. The second is how many chunk files each read call actually opens. The store is chunked at 512×512 pixels and the dataloader draws 256×256 patches at random positions, so a patch usually crosses a chunk boundary and a single read touches more than one chunk file; on average it touches about 2.25 of them (the geometry is derived in Section 4.7.1). The two numbers multiply

to about $41 \times 2.25 \approx 93$ chunk-file opens per sample. A direct count over one thousand random samples measured 92.7. This is the per-sample fan-out the dataloader pays on every sample, whatever the size of the store.

The bytes behind those opens are small. The compressed chunk sizes on the production store, measured over one hundred random chunks for each of twenty-three variables, are heavily bimodal: most variables have a median chunk at the codec’s compression floor (about 4 kilobytes), the all-fill ocean chunks that the codec squeezes to almost nothing, while land chunks run from tens of kilobytes up to about 1.5 megabytes for the largest static fields. At the mean chunk sizes the compressed payload pulled per sample is on the order of three megabytes, rising to about twenty megabytes when a patch lands on a worst-case land region. The output tensor the dataloader returns, after stacking, transposing, and casting, is 9.5 megabytes per sample. The numbers that matter for the rest of the chapter are these: about ninety chunk-file opens per sample, on the order of three megabytes of compressed bytes, and 9.5 megabytes of materialised tensor.

4.6.3 Bandwidth ceiling and the observed gap

The parallel filesystem’s bandwidth on a single MareNostrum 5 ACC node was measured directly. A synthetic block-copy benchmark with one-megabyte reads, with the operating system’s page cache enabled, reports about 556 megabytes per second from a single stream. Twelve parallel streams over separate four-gigabyte files reach an aggregate 2510 megabytes per second, which is roughly 209 megabytes per second per stream.

If the per-sample read were bandwidth-bound, three megabytes per sample at 556 megabytes per second would take about five milliseconds. The observed per-sample fetch is about 1.25 seconds, roughly two hundred times slower than the bandwidth ceiling. The two-hundred-fold gap is not in the bytes. It is in the per-chunk overheads identified above, foremost the metadata latency paid to open each chunk file. The remainder of the subsection isolates that metadata cost directly.

4.6.4 Metadata cost as a function of file count

A synthetic probe was built to isolate the per-open cost on the parallel filesystem. The probe writes N files of size S into a single directory, then reads each one back either in order or in shuffled order, and records the wall time per open.

The first result is that the cost is charged per file opened, not per byte read. With the operating system’s metadata cache warm, each open of a file up to about 100 kilobytes costs approximately 0.11 milliseconds, and this is flat as the file count grows from ten to ten thousand. Holding the total payload fixed and splitting it across more files makes the read dramatically slower: 100 megabytes in one file takes 44 milliseconds, while the same 100 megabytes in ten thousand 10-kilobyte files takes 1082 milliseconds, about 25 times longer. What the dataloader pays for is the number of files it opens, not the volume of data.

The 0.11-millisecond figure is the warm cost, paid when the file’s metadata is already cached. The cold cost, paid on a cache miss, is very different, and it is the one that matters for our dataset. A second measurement reads ten thousand random files from each of two single-directory pools, one of 100 thousand files and one of one million files (all 10 kilobytes), in interleaved shuffled order so neither pool is favoured by the cache.

On a cache miss each open costs about 20 milliseconds at the median and 23 at the 95th percentile, matching the 19 to 20 milliseconds per chunk measured on real Zarr chunks. The two pools agree to within 8 per cent at every percentile. So a cold open costs roughly two hundred times a warm one, and, importantly, that cold cost does not grow with the number of files in the directory: a directory of one million files is no slower per open than one of one hundred thousand. Whether an open is warm or cold is set by cache occupancy, not by directory size. Section 4.6.6 returns to which regime the real stores operate in.

A further measurement verifies that the synthetic probe is a faithful proxy for the real store. When the synthetic files' byte sizes are sampled from the real chunk-size distribution rather than being held at a single size, the synthetic per-open wall and the real per-open wall agree at every percentile. For the purposes of the diagnosis, the synthetic probe is representative of the real store.

4.6.5 Production-shape simulation

The measurements so far show that each open pays a metadata latency whose cold cost is about 20 milliseconds and does not depend on directory size. The next step is to confirm, at production scale and under realistic concurrency, that changing only the number of opens per sample changes the per-sample time. Two synthetic stores were built, each at roughly 30 gigabytes total, that differ only in their per-sample file-open pattern. Store A mimics the single-zarr layout: 23 subdirectories of about thirteen thousand files each, with sizes sampled from the real chunk distribution, and a per-sample read pattern that opens 40 randomly chosen files. Store B mimics a hypothetical alternative layout that collapses the per-sample file count: one subdirectory of thirteen thousand files, each about 2.3 megabytes, with a per-sample read pattern that opens 2 randomly chosen files. The total bytes are matched between the two stores; only the number of opens per sample differs. Twelve concurrent workers read one hundred samples each in alternating access patterns (A then B then A, and A then B then A then B) to control for the warming effect of repeated reads on either store.

Table 7 reports the cold-cache results across two independent measurements. Store A pays approximately 660 milliseconds per sample (about 16 to 17 milliseconds per opened file). Store B pays about 90 milliseconds per sample (about 45 milliseconds per opened file, of which most is the byte transfer of the 2-megabyte file rather than metadata latency). The per-sample cost is about seven times lower under the merged-layout simulation. The drift between consecutive passes (about 25 per cent on the A side, about 8 per cent on the B side) is small enough to confirm that the measurement is repeatable.

Pass	Store A (ms)	Store B (ms)	A/B ratio
First measurement, pass 1	669	94	7.16×
First measurement, pass 2	524		
Second measurement, pass 1	668	89	7.55×
Second measurement, pass 2	536	82	6.56×

Table 7: Per-sample wall time on the production-shape simulation, twelve concurrent workers, one hundred samples per worker, cold cache. The seven-fold A-over-B speed-up is robust across two independent measurements and across consecutive passes within each measurement.

A separate experiment repeats the same A/B comparison at a much smaller scale: about 31 000 files for Store A and about 1350 files for Store B, total under three gigabytes for each. At this scale both pools fit comfortably in the operating system’s file-metadata cache (described in the next subsection). Inside a single job, Store A drops from 39 milliseconds per sample on the first pass to 8.6 milliseconds on the second pass, a 4.5-fold warming, while Store B stays at about 16 milliseconds. After warming, Store A is faster than Store B, because the per-open cost has collapsed to about 200 microseconds and Store B still has to transfer 2 megabytes per opened file. The cross-over confirms that the seven-fold speed-up at production scale is driven by cache misses on the metadata path, not by an intrinsic property of the file-count difference.

4.6.6 Cause of the store-size effect on read performance

This brings us back to the second open question: why the per-sample read time grows with the total on-disk size of the store, even though a sample reads the same small slice either way. The cold-regime measurement above already rules out one explanation, that a bigger directory makes each open intrinsically slower, since the 100-thousand and one-million-file pools cost the same per open. The explanation the measurements do support is cache occupancy.

Every chunk file is opened by name. To turn a name into an on-disk location the operating system walks the path through a series of directory entries and caches the results in the Linux *dentry cache*, the in-kernel store of recently-resolved path components [25], along with the matching file-identifier records (the *inodes*). That cache holds only a bounded number of entries. An open whose record is still cached is served from memory in about 100 microseconds; an open whose record has been evicted pays the full metadata latency, about 20 milliseconds. The per-miss cost is essentially constant, and does not grow with the size of the store, as the cold-regime measurement above confirmed.

What grows with the size of the store is the *rate* of misses. The dataloader reads chunks at random positions, so each open is close to a random draw from the store’s chunk files. The cache holds only a fixed number of records, so the more distinct chunk files a store has, the smaller the fraction that fits in cache, and the more often a random open misses. A store with more files therefore pays the 20-millisecond miss cost on a larger share of its opens. That raises the mean per-open time, and with it the per-sample time, even though each sample still reads the same few small slices. The two single-zarr stores fit this picture cleanly, since they differ in nothing but size: the 213-gigabyte store holds about 2.45 million chunk files against about 710,000 in the 66-gigabyte one, a factor of about three and a half, and its slow batches are about three times larger, the rough proportionality a miss rate that tracks file count would produce.

The multi-zarr layout the AI4Land team had been using points the same way. It holds about 6.7 million chunk files (Table 1), about nine and a half times the short single-zarr store, and its slow batches were about eight times larger, so the slowdown rises roughly in step with the file count across an order of magnitude. This is the best-supported explanation we can give, and it is consistent with every store we measured, but we did not measure the cache-miss rate directly, so the mechanism that links file count to slowdown stays inferred rather than measured; the Future Work section returns to it. The robust, layout-independent fact is the one the rest of the chapter builds on: how expensive each open is depends on cache state, but the number of opens per sample is what an

optimization can change directly.

4.6.7 Summary of evidence

The chain of measurements rules out every alternative explanation and leaves one mechanism standing. The bottleneck is not the preprocess step (the Phase 2 split and the preprocessed-variant check). The bottleneck is not the Zarr synchronizer (the three comparisons of the previous subsection plus the source-level inspection of the library). The bottleneck is not the codec or the decompression step (1 per cent of the per-chunk cost). The bottleneck is not the bytes themselves (around three megabytes per sample against a 556-megabytes-per-second single-stream ceiling). The bottleneck is not the inter-process queue, the worker count, or any other coordination layer (the linear scaling of Phase 3).

The bottleneck is the cost of opening each chunk file on the parallel filesystem. Two quantities set the per-sample cost: how many chunk files a sample opens, and how much each open costs. The count is large, about ninety opens per sample on the single-zarr layout, and it is fixed by the data layout, not by the hardware. The cost of each open swings between about 100 microseconds when the file’s metadata is cached and about 20 milliseconds when it is not, and which regime applies depends on how much of the store’s metadata fits in cache; this second factor is what produces the store-size effect. The count is the factor an optimization can attack directly, and it is the larger lever: even if every open were served warm, ninety opens per sample is far more work than is needed. This is why the next subsection cuts the count by merging the variables, rather than trying to keep the store resident in cache. Reducing the number of opens helps in every cache state, whereas keeping a multi-hundred-gigabyte store warm is not something the dataloader can guarantee. Cutting the count also cuts the per-chunk Python work in the xarray layer, which is the cost that remains once the opens are few; the Future Work section returns to it.

A natural question is which of these regimes the pipeline operates in for a real run. The answer is the cold one. A production training pass sweeps a full-size store, whose working set of chunk files is far larger than the operating system can hold in cache, so nearly every open misses and pays the full latency. The partial warming seen on the smaller store is the store-size effect itself: it shows up only because that store is small enough for part of its metadata to stay resident, which a production run cannot count on. The operationally relevant cost is therefore the cold open count, and cutting that count is what the next subsection sets out to do.

The literature on parallel filesystems is consistent with this picture. The historical design of the General Parallel File System optimises for high large-file bandwidth and routes all metadata operations through dedicated metadata servers that become a bottleneck when the system holds millions of small files [29]. An empirical study of file-create performance on the same storage architecture, IBM Spectrum Scale (the current product name for GPFS), finds that metadata operations such as file creation scale poorly, and that a small fraction of files account for most of the total create time, a tail-heavy pattern that mirrors the read-time result here [39]. Guidance on storing big scientific data points the same way: it favours a modest number of sensibly sized chunks over a split into millions of tiny ones, precisely the many-small-files pathology the AI4Land production store exhibits [1, 10].

4.7 From a Single Store to a Merged Store

4.7.1 Design of the merged store

The merged store retains the single-zarr layout’s chunk shape (512×512 pixels in space, one timestep along time) and the dataloader’s patch size (256×256 pixels). The change is along the variable axis. The fourteen LUH2 variables, the population variable, and the Köppen–Geiger zone are stacked into a single *dynamic* array, with the variable as a new axis. The six static variables are stacked into a single *static* array. Categorical variables (Köppen–Geiger) are stored as floating point in this representation and cast back to integers at read time. The same bytes that were previously spread across about ninety chunk files per sample now live in a small handful of files, because one chunk file in the merged store contains all the dynamic (or all the static) variables at that spatial chunk and that timestep.

This reduction has a concrete size signature. The merged store holds the same 2000–2015 data as the short single-zarr store in about the same space, roughly 62 gigabytes, but as far fewer and far larger files. Its mean chunk file is about 1.5 megabytes, against the kilobyte-scale chunks of the single-zarr layout, and the whole store is about 43,000 chunk files against roughly 710,000 for the same data unmerged, a sixteen-fold reduction in the number of files. That drop in file count is the direct remedy for the many-small-files pathology that guidance on storing big scientific data warns against [1, 10], and it is the regime the diagnosis pointed to.

The per-sample file-open count on the merged store is computed directly from the geometry. The argument has two parts: how many chunk files one read of the merged store touches on average, and how many such reads each sample issues.

The first part is the chunk-straddling math. The merged store is chunked at 512×512 pixels in space; the dataloader draws 256×256 patches at uniformly random spatial positions. The probability that the patch fits inside one chunk along a single spatial axis is 0.5, because the chunk has 256 pixels of slack before the patch crosses the next chunk boundary. The probability that the patch fits inside one chunk in both axes simultaneously is $0.5 \times 0.5 = 0.25$. With probability 0.5 the patch straddles exactly two chunks (because it crosses one boundary), and with probability 0.25 it straddles four chunks (because it crosses two boundaries). The expected number of chunks the patch overlaps is therefore $0.25 \times 1 + 0.5 \times 2 + 0.25 \times 4 = 2.25$. Every read of the merged store at a single timestep opens, on average, 2.25 chunk files.

The second part is how many reads the dataloader issues per sample. The stacked dynamic array is read at the timesteps the autoregressive training sequence spans, and the stacked static array is read once, four reads in all. At about 2.25 chunk files per read this comes to roughly nine opens per sample, and a direct count over one thousand random samples measured 9.05.

Against the single-zarr layout’s measured 92.7 opens per sample, the merged layout’s 9.05 is a reduction of about a factor of ten. This is the lever the diagnosis identified as the largest available payoff, and, as the previous subsection argued, it cuts the per-sample open count itself, so it pays off in every cache state rather than relying on the store staying warm.

4.7.2 Parity check

Before measuring performance, the merged store was verified to produce samples bit-identical to the single-zarr store. Thirty-two samples were drawn from each store with the same random indices, the five output tensors per sample were compared statistic by statistic (mean, standard deviation, minimum, maximum, number of distinct values, 25th percentile, 75th percentile), and the union of categorical values was checked for set equality. The comparison returned no mismatches at a tolerance of one part in ten billion, and the categorical value sets agreed exactly. The model trained on the merged store therefore sees exactly the data it would see on the single-zarr store; any performance difference attributed to the merge is a property of the storage layout, not of a silent data change.

4.7.3 Per-sample performance

The standalone dataloader profiler was rerun against the merged store across three independent jobs spanning two different cache states. The single-thread per-sample fetch on the merged store has mean times of 360, 201, and 194 milliseconds, against 1550, 1411, and 1385 milliseconds on the single-zarr store under the same conditions. The ratio between single-zarr and merged is approximately seven across all three jobs (medians 7.2, 8.9, 7.2). The seven-fold per-sample speed-up is robust across cache states; the variance in absolute numbers reflects the state of the parallel filesystem and the operating system page cache at the time of the run.

The profiler’s worker-count sweep, run with the model out of the loop so that the figures are pure dataloader delivery rates, returns a similarly clean picture (Table 8). At twelve workers the merged dataloader delivers about seventy samples per second, against about nine for the single-zarr layout, the same sevenfold gap, and the rate grows close to linearly with the worker count. The same lever, per-sample fetch time, that limited the single-zarr layout is now seven times smaller; the worker scaling is unchanged.

Workers	Single-zarr (samples/s)	Merged (samples/s)
1	0.8	5.9
4	3.3	25.3
12	8.7	70.5

Table 8: Dataloader delivery rate (samples per second) with the model out of the loop, single-zarr versus merged, eighty batches per configuration. At twelve workers the merged dataloader delivers about seventy samples per second, roughly eight times the single-zarr layout, and more than a single GPU consumes during training.

The twelve-batch slow-batch pattern that dominated every earlier measurement is structurally absent on the merged store. Each prefetch refill now opens about nine files instead of about ninety, and the cost of the refill is small enough that it no longer produces a visible slow batch in the per-batch wall-clock log. The consequence is the one that matters for training, and it is worth stating as a rate. A single H100 running the AI4Land U-Net consumes about 53 samples per second, the per-GPU compute ceiling derived in the end-to-end measurements below (Section 4.7.5). At twelve workers the single-zarr dataloader delivered about nine samples per second, roughly six times slower than the GPU could train on them, so the GPU starved. The merged dataloader delivers about

seventy, just above the GPU’s rate, so the data side now keeps up. That reversal, from about six times slower than the GPU to slightly faster, is what moves the bottleneck off the data pipeline. The next two subsections put the model back in the loop and confirm it.

4.7.4 Per-step breakdown of the merged per-sample fetch

A separate measurement breaks down the merged per-sample fetch step by step, to understand what the residual 175-millisecond median per sample contains. Each helper inside the dataset code is wrapped in a high-resolution timer and the resulting distribution is collected over one hundred samples. The median breakdown is reported in Table 9.

Step	Median (ms)	Share
Dynamic-variable read and materialise	90	51 %
Static-variable read and processing	60	34 %
In-memory continuous-tensor assembly	15	9 %
Lazy index construction (deferred read)	1.4	1 %
Other per-source extracts	<1	<1 %
Sum of parts	167	96 %
Total per-sample fetch (median)	175	100 %

Table 9: Per-step breakdown of the merged per-sample fetch, one hundred samples, single thread. The dynamic read and the static read together account for 85 per cent of the budget. The lazy index construction is essentially free.

Two observations follow. The first is that the dynamic-variable read and the static-variable read together account for 85 per cent of the residual time. Any further optimization targeted at the residual must attack both; leaving the static side on the current path forfeits a third of the potential gain. The second is that the lazy index construction that builds the deferred-read graph is essentially free at about 1.4 milliseconds. The cost is not in the graph construction itself; it is in the materialise step that follows, where the xarray graph work, the bytes-from-disk layer, and the deferred-evaluation engine all run together. This breakdown informs the Future Work section below.

4.7.5 End-to-end training

The standalone profiler measures the dataloader in isolation. The end-to-end gain that matters for the production training loop requires putting the model back into the loop, where the right measure is wall-clock time, not dataloader delivery rate. The next set of measurements does so on a single MareNostrum 5 ACC node, with identical model hyperparameters, batch size eight, accumulation four, twelve workers per rank, and prefetch factor four, varying only the dataset between single-zarr and merged.

A short run executes fifteen training steps, one validation step, and one test step per epoch, three repetitions per layout. The mean total wall time across the three repetitions is 118.9 seconds on the single-zarr layout and 21.2 seconds on the merged layout, a 5.61-fold end-to-end speed-up. The breakdown is 4.64-fold on the training phase and 7.69-fold on the validation phase. The inflated validation ratio is an artefact of the single-step validation, in which the cost of filling the prefetch queue dominates the measurement on the single-zarr side.

A longer run executes two hundred training steps, thirty validation steps, and thirty test steps per epoch, three repetitions per layout. The mean total wall time across the three repetitions is 354.1 seconds on the single-zarr layout and 76.5 seconds on the merged layout, a 4.63-fold end-to-end speed-up. The breakdown is 4.65-fold on the training phase and 4.51-fold on the validation phase; the validation phase is now properly amortised across thirty steps and reports the same ratio as the training phase. The 4.63-fold ratio is the end-to-end speed-up of the merged layout over the single-zarr layout under the production training configuration.

The per-batch behaviour during the long run is qualitatively different between the two layouts (Figure 4 shows the same repetition of each). Both share a fast-batch floor of about 0.15 seconds, the time the model takes to process one batch when its data is already in hand. On the single-zarr layout that floor is interrupted by the structural twelve-batch worker-cycle stall: about twenty slow batches across the epoch, the largest near 14 seconds. On the merged layout the floor is essentially uninterrupted, with only three small spikes (the largest about 4.5 seconds) and no rigid cadence. The stall pattern that dominated single-zarr training is eliminated, not merely shrunk. (One slow batch the merged layout does not avoid is the first of each epoch, when the dataloader respawns its worker processes; it is a once-per-epoch cost, small in the overall budget, and we left it alone. The single-epoch traces here start after that cold batch.)

These end-to-end numbers confirm the reading from the isolated dataloader, and they pin down the limit that remains. Once the store is merged, the dataloader keeps up with the GPUs and the wall time is set by the model’s own forward and backward passes. The fast-batch floor makes this concrete: a batch of eight samples takes about 0.15 seconds of compute, so one GPU can push at most about $8/0.15 \approx 53$ samples per second. The merged layout at one rank already runs at about 52 samples per second (Table 10 below), which is that ceiling: the dataloader is now feeding the GPU as fast as it can compute. Until the per-batch compute is made cheaper, about 53 samples per second per GPU is the limit, and further data-side optimization buys nothing. The bottleneck has moved from the data pipeline to the GPU.

4.7.6 Scale-out behaviour

The end-to-end DDP measurements report behaviour on a single MareNostrum 5 ACC node. The remaining question is how the layout behaves as the rank count grows from one to eight, since the production training is expected to scale across multiple ranks and possibly multiple nodes.

The sweep covers every combination of rank count (1, 4, 8) and worker count per rank (8, 12, 20), eighteen configurations in all, each run five times. The runs are arranged in a single dependency chain that alternates the dataset between consecutive jobs, so that no two adjacent jobs read the same store and the operating-system cache cannot persist across jobs of the same dataset. For each configuration we discard the highest and the lowest of the five repetitions and average the remaining three. The metric here is samples *trained* per second, that is, samples pushed through the complete training step with the model in the loop, not the dataloader delivery rate of the previous subsections; it is computed from the per-epoch wall time, because the single-zarr layout’s tail of slow batches makes the median per-batch delta a poor estimator of actual throughput.

Three results stand out. First, the merged layout is far faster at every scale: at eight

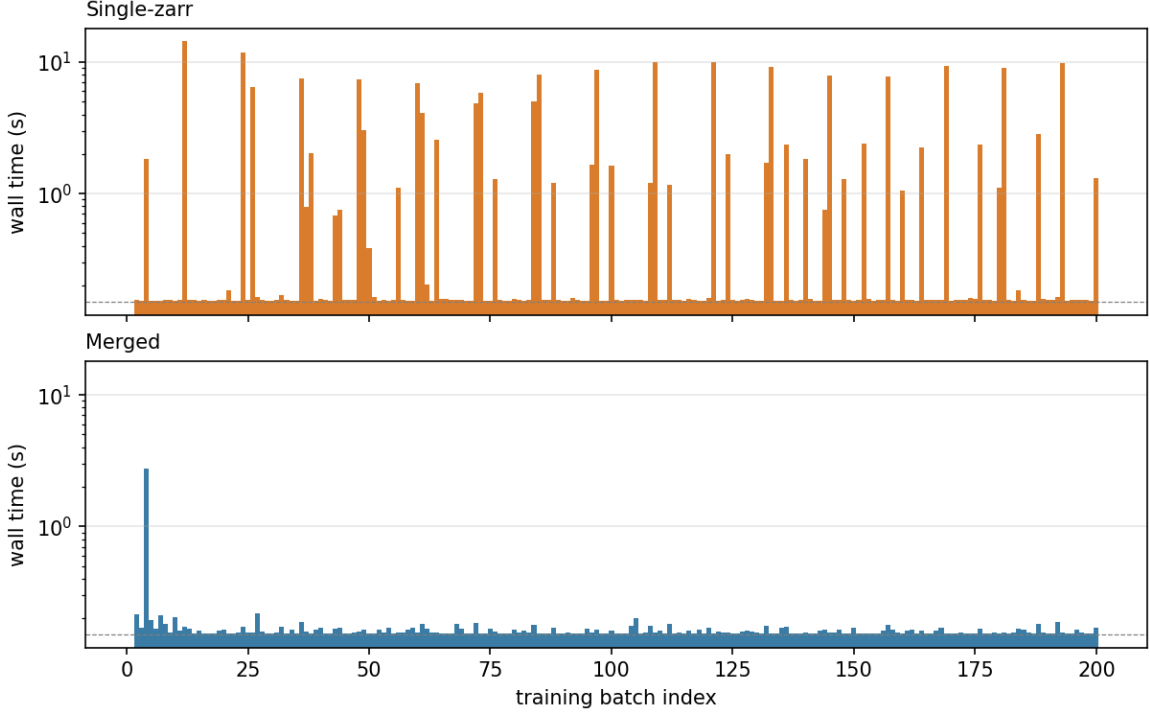


Figure 4: Per-batch wall-clock time during one 200-step training epoch, the same repetition for the single-zarr layout (top) and the merged layout (bottom), on a logarithmic time axis. Both share a fast-batch floor near 0.15 seconds (the model’s forward and backward pass on one batch). The single-zarr layout adds periodic twelve-batch stalls up to about 14 seconds; the merged layout stays near the floor apart from three small spikes.

ranks it reaches about 360 samples trained per second against the single-zarr layout’s best of about 71, and at matched configurations the gain runs from about five-fold (eight ranks, twenty workers) to about nine-fold (eight ranks, eight workers).

Second, the rank axis is *weak scaling*: each added GPU brings its own slice of work, so the total work grows with the resources. The merged layout scales across this axis almost linearly, keeping about 86 per cent of ideal from one to eight ranks (a factor of about 7 out of 8), against about 69 per cent for the single-zarr layout. The single-zarr layout loses about a third of its scaling to contention on the shared metadata server as more ranks read at once; the merged layout, issuing about ten times fewer opens, presses that server far less.

Third, the worker axis at a fixed rank count is the *strong-scaling* axis, and here the two layouts behave oppositely. The single-zarr layout keeps speeding up as workers are added (at eight ranks, 38, then 53, then 71 samples per second for 8, 12, and 20 workers), because it is still dataloader-bound and more workers fetch more data. The merged layout barely moves (about 360 across 8, 12, and 20 workers at eight ranks), because it is already compute-bound at about 53 samples per second per rank (the ceiling established in the previous subsection): eight workers are enough to keep the GPUs fed, and adding more cannot make the GPUs compute faster. This is the same conclusion the isolated dataloader pointed to, now visible end to end. The merged layout has moved the bottleneck off the data pipeline and onto the GPUs, and the residual loss from perfect weak scaling, the shared metadata server under many concurrent ranks, is the one data-side cost left to

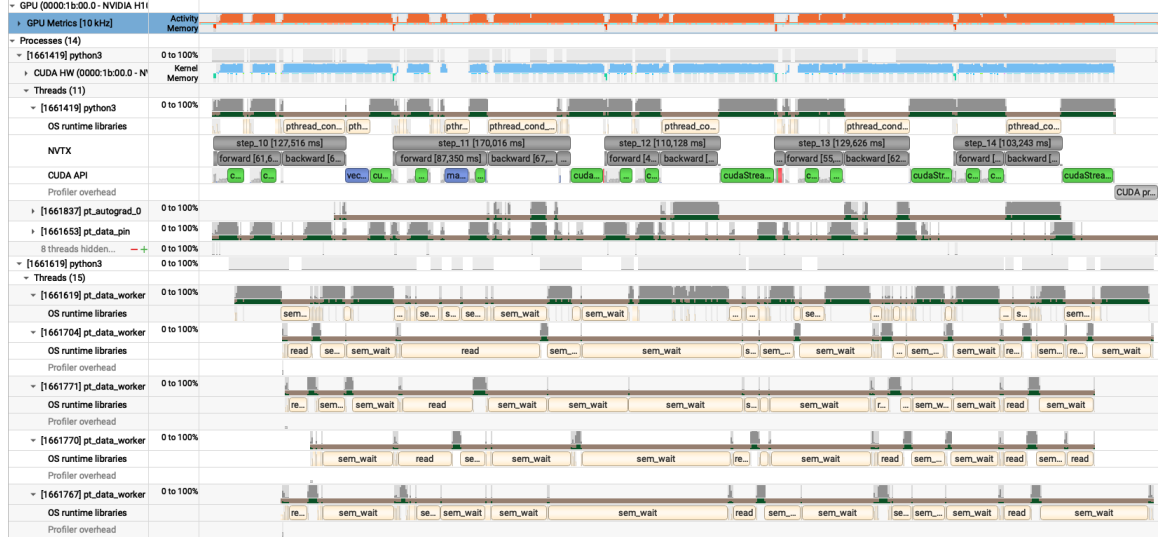


Figure 5: Nsight Systems trace on the merged layout, to be compared with the multi-zarr trace of Figure 2. The GPU track is now busy from one step to the next, the dataloader phase has shrunk to a thin sliver between steps, and the long per-step stalls are gone: the GPU, not the data pipeline, now paces the training step.

chase. The Future Work section returns to it.

4.8 Projected speed-up over the multi-zarr baseline

The end-to-end and scale-out measurements above compare the single-zarr and merged layouts over the short window they share. The production training the AI4Land team had actually been running used the multi-zarr layout over its full, far longer window (Table 1), and we could not repeat the comparison at that scale. What the merge would do for the team’s real training is therefore an extrapolation, but a well-grounded one, because the merged data path has throughput to spare: with the model out of the loop the merged dataloader delivers about seventy samples per second at twelve workers (Table 8), more than the roughly fifty-two a single H100 can train on, and with twenty workers the trained throughput sits flat at that GPU ceiling (Table 10) instead of rising. The data side already runs ahead of the GPU.

Growing the merged store to the full window would not change that. It would raise the file count by roughly the factor that separates the short single-zarr store from the multi-zarr one, about nine and a half, from roughly 43,000 files to roughly 400,000, still fewer than the short single-zarr store’s 710,000, and a sample would still open only about nine of them. Even if all nine missed the cache at about 20 milliseconds each, the opens would cost only about 180 milliseconds per sample, and a store that small would not sit fully cold to begin with. The per-sample time would stay low enough for the dataloader to keep feeding the GPU, so merged training at the full window would remain GPU-bound at about 0.15 seconds per batch, exactly as at the short window.

The multi-zarr baseline, by contrast, is bound by the data loader, and its cost can be reconstructed from the periodic stall. The team’s production run is ten epochs of 2,500 training and 500 validation batches each, plus 500 test batches at the end, about 30,500 batches in all. On the multi-zarr layout one batch in twelve is a slow batch of about 75

Ranks	Workers	Single-zarr			Merged		
		Samp/s	Wall (s)	Start (s)	Samp/s	Wall (s)	Start (s)
1	8	6	2447	58	50	397	104
	12	9	1774	96	51	392	98
	20	13	1327	95	52	408	105
4	8	19	2976	121	167	486	120
	12	28	2106	69	196	434	115
	20	37	1785	80	192	454	114
8	8	38	3038	112	367 [†]	453	120
	12	53	2213	82	364 [†]	436	100
	20	71	1849	105	358	454	82

Table 10: Per-configuration throughput (samples trained per second), total job wall time, and startup time, across rank counts and worker counts per rank, for the two layouts. Each configuration was run five times; for each we drop the fastest and the slowest repetition and average the rest. Total samples grow with the rank count (each rank processes the same per-rank workload), so wall time does not fall as ranks are added; throughput is the comparable figure. [†] A node failure reduced these two configurations to four repetitions; after trimming, the survivors are still cache-warming and bias the mean low, so the warm steady-state value is shown instead.

seconds (the cadence is set by the twelve worker processes, as in the initial profiling), which puts about 2,540 slow batches across the run at roughly 52.9 hours, with the remaining fast batches adding about 1.2 hours, some 54 hours in total. This lands on the roughly 55 hours the team’s full training actually took, the check that makes the rest of the estimate trustworthy. The same 30,500 batches on a GPU-bound merged layout, at 0.15 seconds each, take about 1.3 hours. The merged layout would therefore cut the full training from about 54 hours to about 1.3, a speed-up of order forty-fold (about 42 from these numbers) that removes some 53 of the 54 hours. This is a projection, not an end-to-end measurement: it assumes the full-window merged store stays GPU-bound, and it is tied to the twelve-worker configuration the team used, since a larger worker count would make the slow batch rarer (one in twenty workers rather than one in twelve) and shrink the multi-zarr penalty, and the speed-up with it. The runs that would settle it are set out in the Future Work section.

5 Conclusions

This thesis set out to make AI4Land training computationally more efficient on MareNostrum 5 without changing what the model learns. The starting point was a pipeline that left its H100 graphics processing units idle more than ninety per cent of the time, waiting on data. Profiling traced that wait to the data loader, and the data loader’s cost to a single mechanism: the metadata latency the parallel filesystem pays to open each of the many small chunk files a Zarr store is made of. The diagnosis held up against every alternative we could test, ruling out preprocessing, the Zarr synchronizer, the codec, the raw byte transfer, and the worker count, and three independent measurement paths confirmed it.

Measured against the objectives set out earlier, the work delivers. Training is much faster on a fixed compute budget. Collapsing the five separate stores into one analysis-ready store, and then stacking the variables so that a sample opens about nine files instead of about ninety, makes a single sample load about seven times faster. Measured against the single-store layout on the same window, a full training run is about 4.6 times faster on one node and the throughput about five times higher across eight GPUs. The long periodic stalls that punctuated every earlier run are gone. The second goal, seeing more of the world in the same wall-clock budget, is enabled but not yet demonstrated: the faster data path gives each epoch room to take in more distinct samples, and the Future Work section sets out the measurement that would quantify it. Accuracy is untouched: every layout change was admitted only after a parity check confirmed that the model receives bit-for-bit the same data. And each speed-up was tied by profiling to the component it came from, rather than asserted, which was the measurement objective.

The most durable result is the insight behind those numbers, not the numbers themselves. For any deep-learning pipeline that reads patches from a chunked store on a parallel filesystem, the layout of that store, how many files it is cut into and how large each chunk is, is the single largest lever on training speed. Careless chunking can cost an order of magnitude, and a store that is fast while small can turn slow as it grows and its metadata outgrows the cache. This was not part of the team’s working assumptions before the present work, where chunking had been treated as a detail fixed once when the dataset was built. It is directly actionable, and not only here: the same principle applies to the other geospatial deep-learning projects across the group, and the ELLIOT foundation-model effort, which plans to reuse this pipeline, inherits both the faster pipeline and the reason it is faster. The two diagnostic tools built along the way, the standalone three-phase dataloader profiler and the synthetic filesystem probe, are reusable on any similar pipeline and are themselves part of what this thesis contributes.

With the data path no longer the constraint, the bottleneck has moved onto the GPUs themselves. At the speeds reached here the data loader keeps up with the hardware, so it is now the model’s own forward and backward passes, not data loading, that set the pace. That is the proper end state for this kind of optimization. Where it leads next is the subject of the final section.

6 Future Work

The data path still has room, even if each further gain now matters less than it would have before the GPU became the limit. The clearest direction is to push the chunking strategy further, since the per-sample open count is the lever the diagnosis identified. Three changes are worth measuring. The first is to merge the dynamic and the static data into a single array, so a sample opens one file per timestep rather than one for each of the two groups. The second is to enlarge the spatial chunks, for example from 512 to 1024 pixels on a side, so that a 256-pixel patch almost never straddles a chunk boundary and the expected number of chunks per read falls back towards one. The third is to fold several years into one chunk along time, so the short autoregressive sequence a sample needs comes from one file instead of several. The three combine, and some can be applied in place, which would also remove the second on-disk copy of the dataset that the merged layout currently requires. Two software levers sit on top of chunk geometry: reading chunks through the raw Zarr path instead of through xarray, which removes the per-chunk Python work that now dominates the merged fetch (about 85 per cent of the residual time) and lets the reads overlap across threads; and moving to the next major version of Zarr, which can overlap the per-chunk metadata latency across many chunks at once [7].

One loose end from the diagnosis is worth closing as well. The best explanation for why a larger store reads more slowly, that random access makes the cache-miss rate climb with the number of files, fits every measurement but was never tested directly. A controlled experiment that varies the file count while holding the access pattern and the per-miss cost fixed would confirm or refute it, and would put the store-size effect on the same firm footing as the rest of the diagnosis.

Two measurements would close the loop on the data side. The first is to build the merged store over the full range of years and run the end-to-end comparison this thesis could only do on the short window. That would replace the roughly forty-fold projection of Section 4.8 with a measured number, and confirm that the merged dataloader stays ahead of the GPU at full scale. The second completes the second objective of the thesis, seeing more of the world. With the full-window store in place, measuring how many samples per second a twenty-worker dataloader delivers, and dividing by the GPU’s per-batch compute time, gives the number of extra samples each epoch can afford at no added wall-clock cost: how many more distinct place-and-year combinations the model can be shown per epoch once the data path is no longer the limit. That figure is the concrete form of the second objective, which the speed-up enables but this thesis did not measure.

The larger opportunity, though, is on the compute side, because that is what now limits training. Making the model’s forward and backward passes cheaper, or running them on better-used hardware, is the next real source of speed-up, and it is a problem other groups have already made headway on. Within our own department, colleagues training machine-learning weather models have had good results with Anemoi [12], an open-source framework led by the European Centre for Medium-Range Weather Forecasts (ECMWF) that covers the whole data-driven modelling workflow, from preparing training datasets through scalable model training to running inference. The techniques its users have found effective for keeping the GPU busy, and the engineering behind its training loop, are a natural place to look for the compute-side gains the AI4Land pipeline now needs. Working out what carries over from Anemoi to AI4Land is, in our view, the most promising

direction beyond this thesis.

References

- [1] Ryan P. Abernathey et al. Cloud-native repositories for big scientific data. *Computing in Science & Engineering*, 23(2):26–35, 2021.
- [2] Astral Software. Ruff: An extremely fast Python linter and code formatter. <https://github.com/astral-sh/ruff>, 2024.
- [3] Astral Software. uv: An extremely fast Python package and project manager. <https://github.com/astral-sh/uv>, 2024.
- [4] Barcelona Supercomputing Center. MareNostrum 5: A pre-exascale EuroHPC supercomputer. <https://www.bsc.es/marenostrum/marenostrum-5>, 2024.
- [5] Michelle Barker, Neil P. Chue Hong, Daniel S. Katz, et al. Introducing the FAIR principles for research software. *Scientific Data*, 9:622, 2022.
- [6] Hylke E. Beck, Tim R. McVicar, Noemi Vergopalan, Alexis Berg, Nicholas J. Lutsko, Ambroise Dufour, Zhenzhong Zeng, Xin Jiang, Albert I. J. M. van Dijk, and Diego G. Miralles. High-resolution (1 km) Köppen–Geiger maps for 1901–2099 based on constrained CMIP6 projections. *Scientific Data*, 10(1):724, 2023.
- [7] Davis Bennet. Accelerating xarray with Zarr-Python 3. <https://www.earthmover.io/blog/xarray-open-zarr-improvements/>, 2025.
- [8] Ben B. B. Booth, Chris D. Jones, Mat Collins, Ian J. Totterdell, Peter M. Cox, Stephen Sitch, Chris Huntingford, Richard A. Betts, Glen R. Harris, and Jon Lloyd. High sensitivity of future global warming to land carbon cycle processes. *Environmental Research Letters*, 7(2):024002, 2012.
- [9] CONCERTO Consortium. CONCERTO: Improved carbon cycle representation through multi-scale models and Earth observation for terrestrial ecosystems. <https://www.projectconcerto.eu>, 2025. Horizon Europe Grant Agreement 101185000.
- [10] Dask Development Team. Dask arrays: Choosing good chunk sizes. <https://docs.dask.org/en/stable/array-chunks.html>, 2024.
- [11] Earthmover. Cloud native data loaders for machine learning using Zarr and Xarray. <https://www.earthmover.io/blog/cloud-native-dataloader/>, 2024.
- [12] ECMWF and partners. Anemoi: A framework for machine-learning weather forecasting. <https://github.com/ecmwf/anemoi>, 2024.
- [13] ELLIOT Consortium. ELLIOT: European large open multi-modal foundation models for robust generalization on arbitrary data streams, 2025. Horizon Europe Grant Agreement 101214398.
- [14] EuroHPC JU. European High Performance Computing joint undertaking. <https://eurohpc-ju.europa.eu/>, 2024.
- [15] Katja Frieler, Stefan Lange, Franziska Piontek, Christopher P. O. Reyer, Jacob Schewe, Lila Warszawski, Fang Zhao, Louise Chini, Sebastien Denvil, Kerry Emanuel, et al. Assessing the impacts of 1.5 °c global warming – simulation protocol of the inter-sectoral impact model intercomparison project (ISIMIP2b). *Geoscientific Model Development*, 10(12):4321–4345, 2017.

- [16] GEBCO Compilation Group. GEBCO 2024 grid. Technical report, British Oceanographic Data Centre, 2024.
- [17] Bettina K. Gier, Manuel Schlund, Pierre Friedlingstein, Chris D. Jones, Colin Jones, Sönke Zaehle, and Veronika Eyring. Representation of the terrestrial carbon cycle in CMIP6. *Biogeosciences*, 21(22):5321–5360, 2024.
- [18] Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>, 2022.
- [19] Nicole Hemsoth. HPC file systems fail for deep learning at scale. <https://www.nextplatform.com/2018/10/09/hpc-file-systems-fail-for-deep-learning-at-scale/>, 2018.
- [20] Stephan Hoyer and Joseph J. Hamman. xarray: N-D labeled arrays and datasets in Python. *Journal of Open Research Software*, 5(1):10, 2017.
- [21] George C. Hurtt, Louise Chini, Ritvik Sahajpal, Steve Froking, Benjamin L. Bodirsky, Katherine Calvin, Jonathan C. Doelman, Justin Fisk, Shinichiro Fujimori, Kees Klein Goldewijk, et al. Harmonization of global land-use change and management for the period 850–2100 (LUH2) for CMIP6. *Geoscientific Model Development*, 13:5425–5464, 2020.
- [22] Awais Khan, Arnab K. Paul, Christopher Zimmer, Sarp Oral, Sajal Dash, Scott Atchley, and Feiyi Wang. HVAC: Removing I/O bottleneck for large-scale deep learning applications. Technical report, Oak Ridge National Laboratory, 2022.
- [23] Lingcheng Li, Gautam Bisht, Dalei Hao, and L. Ruby Leung. Global 1 km land surface parameters for kilometer-scale Earth system modeling. *Earth System Science Data*, 16(4):2007–2032, 2024.
- [24] Lightning AI. Speed up model training. <https://lightning.ai/docs/pytorch/stable/advanced/speed.html>, 2024.
- [25] Linux Kernel Internals. Dentry and inode caches. <https://kernel-internals.org/vfs/dcache-icache/>, 2024.
- [26] Modexa. 8 PyTorch DataLoader tactics to max out your GPU. <https://medium.com/@Modexa/8-pytorch-dataloader-tactics-to-max-out-your-gpu-22270f6f3fa8>, 2025.
- [27] Amirpasha Mozaffari, Marina Castaño, Stefano Materia, and Amanda Duarte. Historical reconstruction and future projection of land surface boundary conditions. In *NeurIPS 2025 Workshop on Tackling Climate Change with Machine Learning*, 2025.
- [28] Amirpasha Mozaffari, Marina Castaño, Stefano Materia, Etienne Tourigny, Oscar Molina-Sedano, Jordi Varela-Agrelo, Dario Garcia-Gasulla, Miguel Castrillo Melguizo, Mario Acosta, and Amanda Duarte. AI4Land: A scalable deep learning framework on MareNostrum 5 for high-resolution land surface boundary condition reconstruction. *Procedia Computer Science*, 2026. Proceedings of the fourth EuroHPC User Days 2026. Preprint available at <https://arxiv.org/abs/2606.11793>.

- [29] Barbara Murphy. GPFS parallel file system explained. <https://www.weka.io/blog/gpfs-parallel-file-system/>, 2020.
- [30] D. J. Newman. Zarr storage specification version 2: Cloud-optimized persistence using zarr. Technical report, NASA Earth Science Data and Information System Standards Coordination Office, 2024.
- [31] NVIDIA Corporation. NVIDIA Nsight Systems. <https://developer.nvidia.com/nsight-systems>, 2024.
- [32] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library, 2019.
- [33] Progressive Robot. PyTorch DataLoader tutorial. <https://www.progressiverobot.com/2026/02/04/pytorch-dataloader-tutorial/>, 2026.
- [34] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241. Springer, 2015.
- [35] Geir Kjetil Sandve, Anton Nekrutenko, James Taylor, and Eivind Hovig. Ten simple rules for reproducible computational research. *PLoS Computational Biology*, 9(10):e1003285, 2013.
- [36] Negin Sobhani et al. Accelerating AI/ML workflows in Earth sciences with GPU-native Xarray, Zarr, DALI, and nvcomp. <https://xarray.dev/blog/gpu-pipeline>, 2025.
- [37] TerraDT Consortium. TerraDT: Digital twin of Earth system for cryosphere, land surface and related interactions. <https://terradt.eu>, 2025. Horizon Europe Grant Agreement 101187992.
- [38] Oriol Tintó Prims, Miguel Castrillo, Mario C. Acosta, Oriol Mula-Valls, Alicia Sanchez Lorente, Kim Serradell, Ana Cortés, and Francisco J. Doblas-Reyes. Finding, analysing and solving MPI communication bottlenecks in Earth system models. *Journal of Computational Science*, 36:100864, 2019.
- [39] Marc-André Vef. *Analyzing File Create Performance in IBM Spectrum Scale*. PhD thesis, Johannes Gutenberg-Universität Mainz, 2016.
- [40] Karina Winkler, Richard Fuchs, M. Rounsevell, and Martin Herold. HILDA+ global land use change between 1960 and 2019 (dataset), 2020.
- [41] Andy B. Yoo, Morris A. Jette, and Mark Grondona. SLURM: Simple Linux utility for resource management, 2003.
- [42] Zarr Development Team. Zarr: An implementation of chunked, compressed, N -dimensional arrays. <https://zarr.readthedocs.io>, 2024.